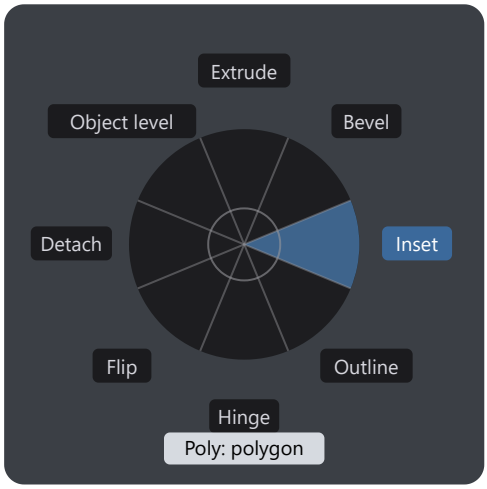
MarkingForge

Reference - every option, every command, tips and troubleshooting

3DS MAX 2027 · VERSION 1.5.65



REAL MARKING MENUS - A QUICK GESTURE NEEDS NO DIAL

MarkingForge works the way marking menus are meant to: the command is chosen by the **direction** of the movement, not by clicking an item. Press the key and flick the mouse towards the command in one quick movement - the command runs and **the dial is not even drawn**. The dial appears only when you hold the key and wait, to remind you where things are. After a few days your hand knows the directions and you stop looking.

Installing is covered by the **Installation Guide**. This document explains how everything works once it is installed: the gestures, the 24 built-in dials, every control of the menu editor, every experimental feature, the configuration file, the colours, the MAXScript commands, tips and tricks, and what to do when something does not behave. To learn it step by step, read the **QuickStart Long** (22 lessons) or the **QuickStart Short** (16 lessons and these tips, about 15 minutes).

Contents

1. Concepts
2. Gestures - everything a dial reacts to
3. Reading the dial
4. The 24 built-in dials
5. The menu editor, control by control
6. Contextual variants (conditions)
7. Content sources
8. Item types
9. Shortcuts and mouse buttons
10. Colours
11. Experimental features
12. The MarkingForge menu
13. Menus carried by scenes
14. The configuration file (menus.json)
15. MAXScript commands
16. Tips and tricks
17. Problems and solutions
18. Files, privacy and safety
19. Support, bug reports and feature requests

1. Concepts

Term	Meaning
Dial	The round menu that appears around the cursor while you hold a key. Eight directions, a centre that cancels, optionally a list of rows under it.
Slot	One of the 24 dials. Slot N is the action *MarkingForge: Menu N* in the Hotkey Editor; its key or mouse button opens it.
Direction	N, NE, E, SE, S, SW, W, NW. Each holds one item - or nothing (an empty direction cancels).
Item	What a direction runs: a 3ds Max action, a macroscript, your own MAXScript, a submenu or a value slider (chapter 8).
List under the dial	Up to twelve rows drawn below the ring for things that do not fit in eight directions. Rows are clicked, not aimed at.
Variant	Alternative contents of the same dial with a condition - the object class, the sub-object level, the window under the cursor... The first variant whose condition matches wins; *Base* is shown when none does (chapter 6).
Content source	Instead of fixed items, a dial can show live data: the modifier stack, the undo list, recent commands, selection sets (chapter 7).
Submenu	A whole dial behind one direction, opened by moving through it without releasing. Its caption ends with **>**. The built-in Create and Modifiers dials have several.
Set (page)	One of several named contents of ONE dial (since 1.0.9) - *Modelling*, *UV*, *Retopo* on the same key. The key opens one of them, the set *on the key*; the others are the dial's further **pages**: turn the mouse wheel while the dial is open (1.2.0). To change which set the key opens: **Put this set on the key** in the editor, a *Next set* > row on the dial, or the action *Menu N - next set*.
Dial file (.mfdial)	One dial in a file of its own - to keep, send or load into another slot. The **dial library** is every dial MarkingForge ships, one such file each (1.1.0, chapter 5).
Hotbox	A slot laid out as tiles of 3ds Max's main menu instead of eight directions.
Default menu	What a slot without its own menu shows. Every slot of the built-in layout has its own menu, so the default only matters for slots you empty yourself.
Caddy	The small settings window 3ds Max shows for Chamfer, Extrude, Inset and similar commands. The dial can open it or skip it (chapter 2).

2. Gestures - everything a dial reacts to

Gesture	What happens	Notes
Hold the key	The dial appears around the cursor.	The centre stays where it opened - the dial does not follow the mouse.
Move towards a direction, release the key	That direction runs.	The wedge and its caption light up while you aim, so you see the choice before committing.
Turn the mouse wheel while the dial is open	The dial shows its next page (towards you) or the previous one - its other sets - at the same place; release over a command to run it.	Since 1.2.0. Only a dial with sets, at its first level, or a hotbox that is one of the sets (1.4.5) - not in a submenu, a search or a dial pinned by typing. One page per notch (a touchpad too, since 1.4.3); the context variant stays the one of where the gesture began, and a queue of picks is dropped when the page turns. The wheel always changes what you see: a page whose variant is the very dial already on screen (a page copied from another dial brings that dial's variants along) shows its own contents instead (1.5.34). The footer says *page 2 of 3 - mouse wheel*; the next press opens the page turned to last, remembered by name in last_pages.json beside menus.json, also across sessions (1.4.8; until 1.4.7 the set on the key). Over a window that has a variant of its own on another page - Track View, the Slate editor, Particle View, Scene Explorer - that page opens instead, and the page you turned to stays remembered for the 3D view (1.5.55). At any other time the wheel zooms and scrolls as always - it is taken only when it turns a page.
Press and move at once (flick)	The direction runs without the dial being drawn.	The fastest way. Works as soon as your hand knows the direction. The context - the object and the window under the cursor - is read where the gesture began, so a flick does exactly what the held dial shows. A flick longer than the distance to the settings rim counts as released past it (since 1.5.2 - until 1.5.1 every flick counted as released inside, so a flick at Extrude always opened its caddy). `MarkingForge.eventLog()` gives each flick's length as *[stroke N px]*.
Tap (press and release without moving)	Nothing, unless the dial has a tap item.	Set in the editor: *A tap runs*. Uses the context variant that matches, like the dial.
Release in the centre	Cancel - nothing runs.	The circle in the middle is the dead zone: 36 px from the centre (18 px before 1.0.9), so it is found without aiming.
Release over an empty direction	Cancel.	An empty direction is drawn with a dashed outline and never lights up.
Esc, or the right mouse button	Cancel and close.	
Let go of the key when 3ds Max misses the release	The dial closes exactly as a release would have closed it - running what you aimed at.	Since 1.4.4. In some states 3ds Max switches its shortcuts off (after starting a Text object, say) and never reports the key coming up. MarkingForge watches the key itself: up for a quarter of a second with no word from 3ds Max, and the dial closes. `MarkingForge.eventLog()` shows a **RELEASE_LOST** line when that happens. It watches the keys of a keyboard dial; a dial opened by a mouse button is not watched this way.

Hotbox: point at a menu title, then at an entry	Release the key - or click the entry with the left mouse button, as in Maya - and the entry runs.	A click closes the hotbox at once; the key's release after it runs nothing more. Hotbox entries leave the selection alone. A right click or Escape (since 1.5.14) closes it without running anything.
Move past the captions (the settings rim)	Reverses the settings choice: an item that normally runs at once opens its caddy, and the other way round.	The rim is the **dotted circle** outside the dial, 20 px beyond the farthest caption by default (since 1.5.2; 30 px from 1.0.9, at the window's edge before) - since 1.5.10 the distance is yours to set, 0 to 200 px, in the editor's **Settings ring...**; it brightens once the cursor is past it. The dots at an item's edge show which way the release will go. Coming back inside before releasing undoes the choice. A flick is measured against the same rim.
Move through a submenu direction	The submenu opens IN PLACE: the dial stays, its tiles become the submenu's.	Up to three levels; the footer shows the path (1.4.9). Nothing is aimed until the cursor is back inside the dial; the centre goes back up a level. Until 1.4.7 the submenu opened as a new dial further out.
Click a row of the list under the dial	That row runs.	Hold the key, move down to the list, point, click.
Cross the ring and come back to the dashed circle	The item is queued (a chain). Releasing in the dashed circle runs the whole queue as ONE undo; releasing at the very centre or over an empty direction discards it. A queued command that switches on a tool (Inset, Extrude, Bevel, Chamfer, Outline) stops the queue until a right-click ends the tool (1.5.44); each such step is its own undo.	Experimental *multiPick* - off by default. The dashed circle round the centre marks the return zone (from the dead zone out to 92 px) and turns green while a queue waits; the dead zone always cancels. The queue shows as a numbered row above the dial.
Tap with no tap item on the dial	Runs the suggested next command, when pair learning knows one and it sits on this dial.	Experimental *pairLearning* - on by default. The suggestion is the outlined tile; an explicit tap item always wins.
Keep moving through a value direction	The distance sets a number; the release commits it - on the object under the cursor, and one Ctrl+Z takes it back (since 1.5.46). Passing through the direction, or giving up in the centre, changes nothing, the selection included.	Experimental *dragValue* - off by default.
Start typing while the dial is open	The dial becomes a search over all 3ds Max actions. It pins itself, so you can let go of the key; Enter runs, Esc closes. Clicking into a text field of 3ds Max (a name, a spinner) closes it too, running nothing, and the	Experimental *gestureSearch* - off by default.

	field gets what you type (since 1.5.47).	

> The dial never takes keyboard focus from 3ds Max. That is why it works over the Slate editor and Track View without changing what they do - and why switching to another program closes it as a cancel.

3. Reading the dial



A dial aimed east.

You see	It means
Dark disc with eight wedges	The eight directions.
Lit wedge and tile	What a release will run now.
Dashed outline	An empty direction - releasing there cancels.
Red caption	A broken item: the action or macro does not exist in this 3ds Max (for example a plugin that is not installed). Releasing does nothing.
Grey caption	Unavailable right now (the command is disabled in this context).
Dot on a tile	A toggle that is on (checked).
> on a tile	A submenu.
... at a tile's edge	The command has a settings dialog (caddy).
Outlined tile	The default direction - what a tap runs (with pair learning: the command you usually run next), and Enter on a pinned dial.
Dotted circle outside the dial	The settings rim - brighter once you are past it (chapter 2).
Dashed circle round the centre	A chain is being built - come back inside it to queue the item.
Footer	The dial's name (for a context variant, the variant's), the object under the cursor the command will act on, on a dial with sets *page 2 of 3 - mouse wheel*, and, when it matters, why the dial is empty.

4. The 24 built-in dials

The layout a fresh installation starts with (**Load default dials...** in the editor brings it back; **Dial library...** restores one dial). The grids show directions as they sit on screen, the dial's name in the centre. Opposite directions hold opposite operations - All / None, Grow / Shrink, Hide / Unhide - so the hand learns one movement and its reverse.

The poly layout, level by level

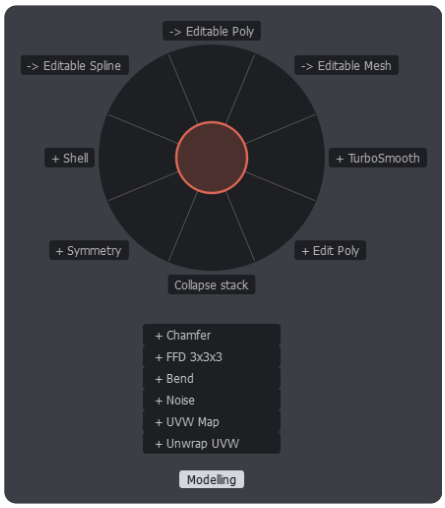
The same on an Editable Poly and under an Edit Poly modifier. NW always goes back to object level.

Level	N	NE	E	SE	S	SW	W	NW
Object	Vertex	Edge	Border	Polygon	Element	Attach	+ TurboSmooth	Collapse stack
Vertex	Chamfer	Weld	Connect	Break	Extrude	Remove	Target Weld	Object level
Edge	Connect	Chamfer	Loop	Ring	Extrude	Weld	Remove	Object level
Border	Cap	Chamfer	Extrude	Loop	Create shape	Shrink	Grow	Object level
Polygon	Extrude	Bevel	Inset	Outline	Hinge	Flip	Detach	Object level
Element	Detach	Hide	Flip	Unhide all	Hide unselected	Shrink	Grow	Object level

Alt+1 Modelling

dial 1

NW -> Editable Spline	**N** -> Editable Poly	**NE** -> Editable Mesh
W + Shell	**Modelling**	**E** + TurboSmooth
SW + Symmetry	**S** Collapse stack	**SE** + Edit Poly



List under the dial: + Chamfer · + FFD 3x3x3 · + Bend · + Noise · + UVW Map · + Unwrap UVW

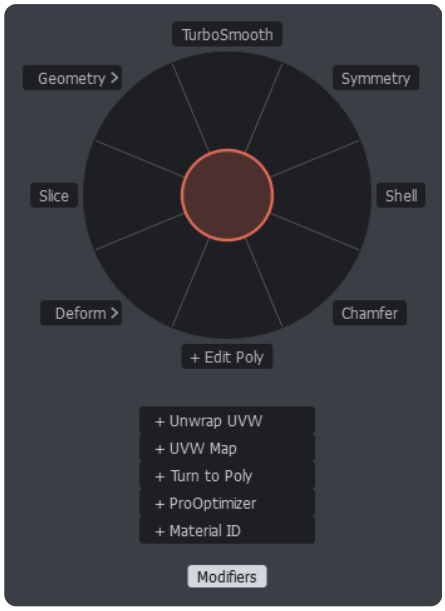
Changes with the situation (first match wins):

- **Materials (Slate)** - when cursor over the Slate Material Editor
- **Track View** - when cursor over Track View
- **Particle View** - when cursor over Particle View
- **Scene Explorer** - when cursor over the Scene Explorer
- **Modifier stack** - when cursor over the command panel
- **UV (Unwrap)** - when Unwrap UVW is current in the Modify panel
- **Edit Poly: vertex** - when Edit Poly is current in the Modify panel, sub-object level: Vertex
- **Edit Poly: edge** - when Edit Poly is current in the Modify panel, sub-object level: Edge
- **Edit Poly: border** - when Edit Poly is current in the Modify panel, sub-object level: Border / Segment
- **Edit Poly: polygon** - when Edit Poly is current in the Modify panel, sub-object level: Polygon
- **Edit Poly: element** - when Edit Poly is current in the Modify panel, sub-object level: Element
- **Edit Poly: object** - when Edit Poly is current in the Modify panel
- **Poly: vertex** - when the object is an Editable Poly, sub-object level: Vertex
- **Poly: edge** - when the object is an Editable Poly, sub-object level: Edge
- **Poly: border** - when the object is an Editable Poly, sub-object level: Border / Segment
- **Poly: polygon** - when the object is an Editable Poly, sub-object level: Polygon
- **Poly: element** - when the object is an Editable Poly, sub-object level: Element
- **Poly: object** - when the object is an Editable Poly
- **Spline: vertex** - when a shape (spline) is selected, sub-object level: Vertex
- **Spline: spline** - when a shape (spline) is selected, sub-object level: Spline
- **Spline** - when a shape (spline) is selected
- **Nothing selected** - when nothing is selected

Alt+2 Modifiers

dial 2

NW Geometry >	**N** TurboSmooth	**NE** Symmetry
W Slice	Modifiers	**E** Shell
SW Deform >	**S** + Edit Poly	**SE** Chamfer



Submenu Deform (SW): N + Bend · NE + Twist · E + Taper · SE + Noise · S + FFD 3x3x3 · SW + Relax · W + Stretch · NW + Skew

Submenu Geometry (NW): N Mirror · NE Array · E Cap Holes · SE Vertex Weld · S Face Extrude · SW + Quadify Mesh · W OpenSubdiv · NW MeshSmooth

List under the dial: + Unwrap UVW · + UVW Map · + Turn to Poly · + ProOptimizer · + Material ID

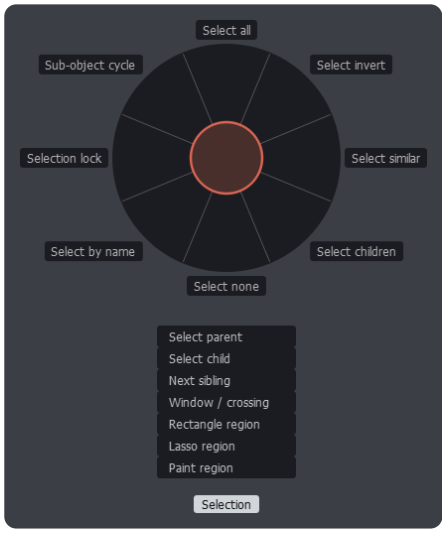
Changes with the situation (first match wins):

- **Shape modifiers** - when a shape (spline) is selected

Alt+3 Selection

dial 3

NW Sub-object cycle	**N** Select all	**NE** Select invert
W Selection lock	Selection	**E** Select similar
SW Select by name	**S** Select none	**SE** Select children



List under the dial: Select parent · Select child · Next sibling · Window / crossing · Rectangle region · Lasso region · Paint region

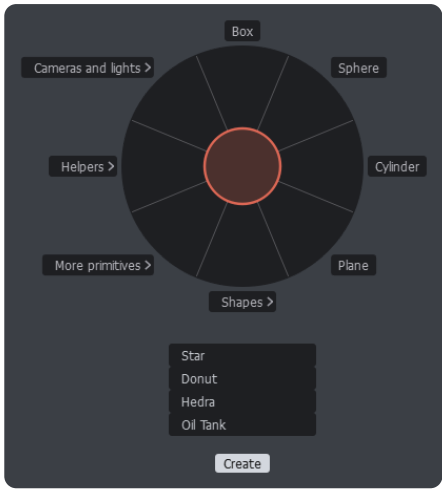
Changes with the situation (first match wins):

- **Selection** - when Edit Poly is current in the Modify panel, sub-object level: object level
- **Edit Poly: selection** - when Edit Poly is current in the Modify panel
- **Selection** - when the object is an Editable Poly, sub-object level: object level
- **Poly: selection** - when the object is an Editable Poly

Alt+4 Create

dial 4

NW Cameras and lights >	**N** Box	**NE** Sphere
W Helpers >	**Create**	**E** Cylinder
SW More primitives >	**S** Shapes >	**SE** Plane



Submenu Shapes (S): N Line · NE Arc · E Circle · SE Ellipse · S Rectangle · SW NGon · W Helix · NW Text

Submenu More primitives (SW): N Cone · NE Torus · E Tube · SE GeoSphere · S Pyramid · SW Teapot · W Chamfer Box · NW Capsule

Submenu Helpers (W): N Dummy · NE Point · E Grid · SE Tape · S Protractor · SW Compass · W Expose Transform · NW Container

Submenu Cameras and lights (NW): N Target camera · NE Physical camera · E Free camera · SE Sun positioner · S Free light · SW Target light · W Omni light · NW Match camera to view

List under the dial: Star · Donut · Hedra · Oil Tank

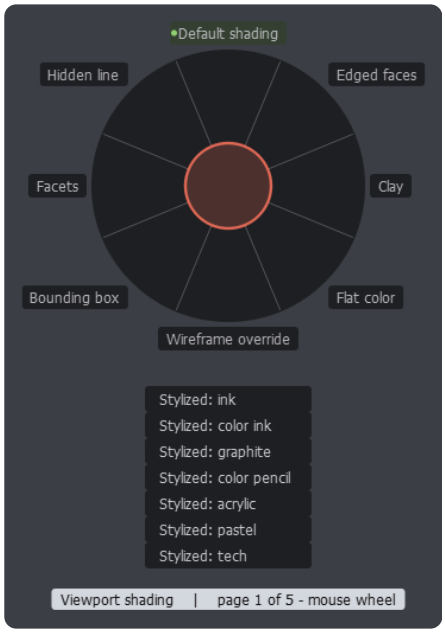
Changes with the situation (first match wins):

- **Create shapes** - when a shape (spline) is selected

Alt+5 Viewport shading

dial 5

NW Hidden line	**N** Default shading	**NE** Edged faces
W Facets	**Viewport shading**	**E** Clay
SW Bounding box	**S** Wireframe override	**SE** Flat color

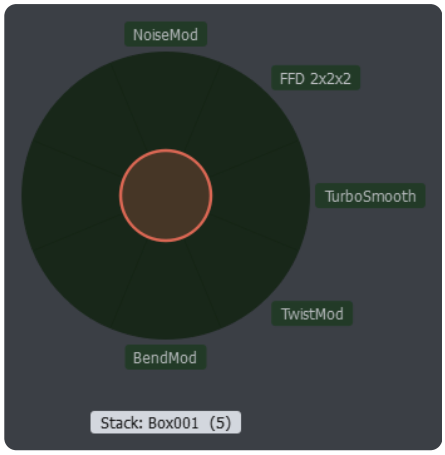


List under the dial: Stylized: ink · Stylized: color ink · Stylized: graphite · Stylized: color pencil · Stylized: acrylic · Stylized: pastel · Stylized: tech

Alt+6 Modifier stack

dial 6

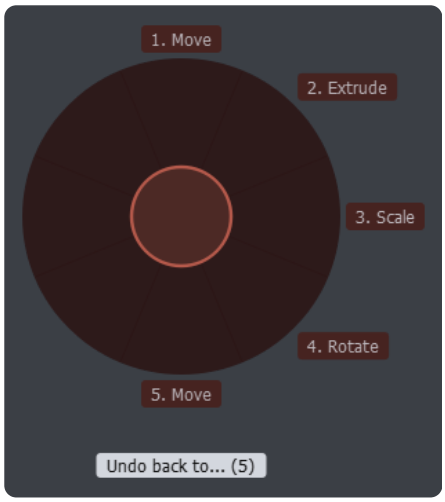
Filled live from the modifier stack of the selected object (top eight).



Alt+7 Undo

dial 7

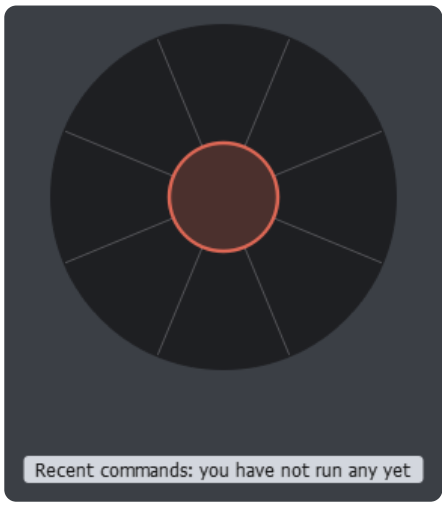
Filled live from Max's undo list - N undoes one step, NE two, and so on.



Alt+8 Recent commands

dial 8

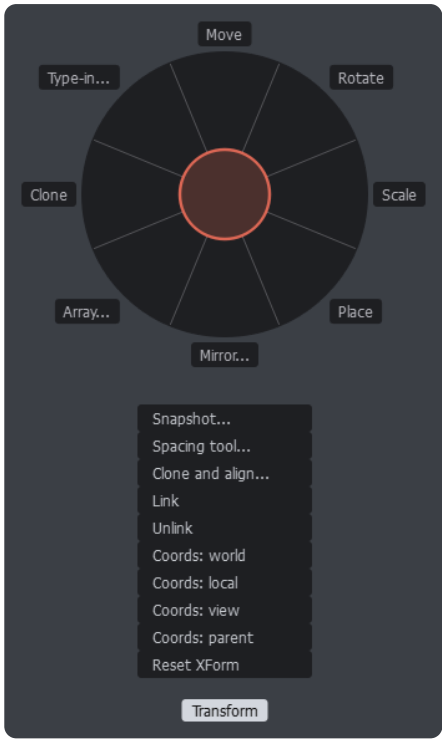
Filled live from the eight commands you ran most recently.



Ctrl+Alt+1 Transform

dial 9

NW Type-in...	**N** Move	**NE** Rotate
W Clone	**Transform**	**E** Scale
SW Array...	**S** Mirror...	**SE** Place

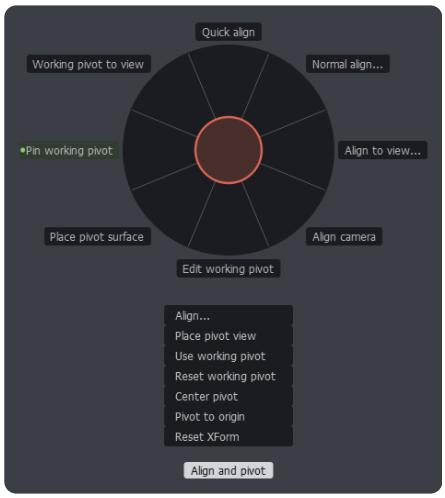


List under the dial: Snapshot... · Spacing tool... · Clone and align... · Link · Unlink · Coords: world · Coords: local · Coords: view · Coords: parent · Reset XForm

Ctrl+Alt+2 Align and pivot

dial 10

NW Working pivot to view	**N** Quick align	**NE** Normal align...
W Pin working pivot	**Align and pivot**	**E** Align to view...
SW Place pivot surface	**S** Edit working pivot	**SE** Align camera

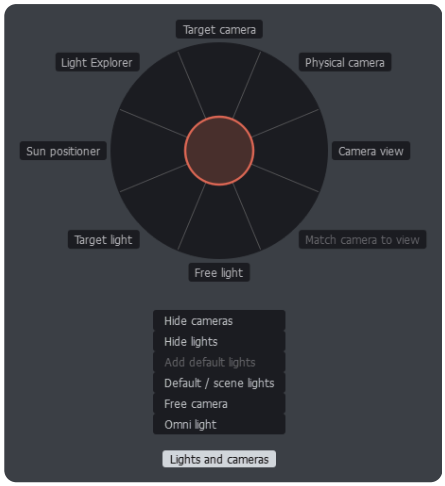


List under the dial: Align... · Place pivot view · Use working pivot · Reset working pivot · Center pivot · Pivot to origin · Reset XForm

Ctrl+Alt+3 Lights and cameras

dial 11

NW Light Explorer	**N** Target camera	**NE** Physical camera
W Sun positioner	**Lights and cameras**	**E** Camera view
SW Target light	**S** Free light	**SE** Match camera to view



List under the dial: Hide cameras · Hide lights · Add default lights · Default / scene lights · Free camera · Omni light

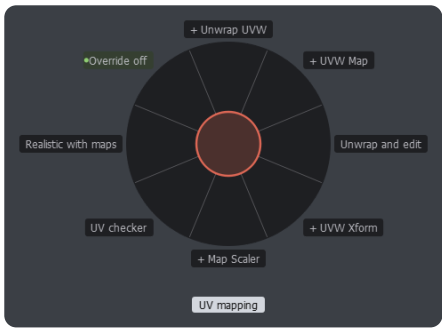
Changes with the situation (first match wins):

- **Camera** - when a camera is selected
- **Light** - when a light is selected

Ctrl+Alt+4 UV mapping

dial 12

NW Override off	**N** + Unwrap UVW	**NE** + UVW Map
W Realistic with maps	**UV mapping**	**E** Unwrap and edit
SW UV checker	**S** + Map Scaler	**SE** + UVW Xform



Changes with the situation (first match wins):

- **UV (Unwrap)** - when Unwrap UVW is current in the Modify panel

Ctrl+Alt+5 Snaps and grid

dial 13

NW Grid front	**N** Snaps	**NE** Angle snap
W Grid top	**Snaps and grid**	**E** Percent snap
SW AutoGrid	**S** Hide grids	**SE** Snap settings...

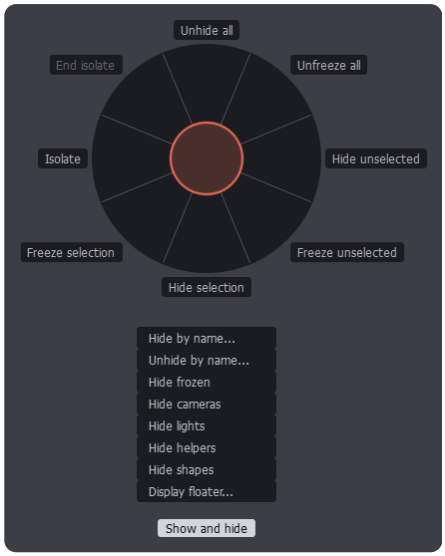


List under the dial: Cycle snap type · Snap to frozen · Axis constraints in snaps · Spinner snap · Grid left · Home grid

Ctrl+Alt+6 Show and hide

dial 14

NW End isolate	**N** Unhide all	**NE** Unfreeze all
W Isolate	**Show and hide**	**E** Hide unselected
SW Freeze selection	**S** Hide selection	**SE** Freeze unselected



List under the dial: Hide by name... · Unhide by name... · Hide frozen · Hide cameras · Hide lights · Hide helpers · Hide shapes · Display floater...

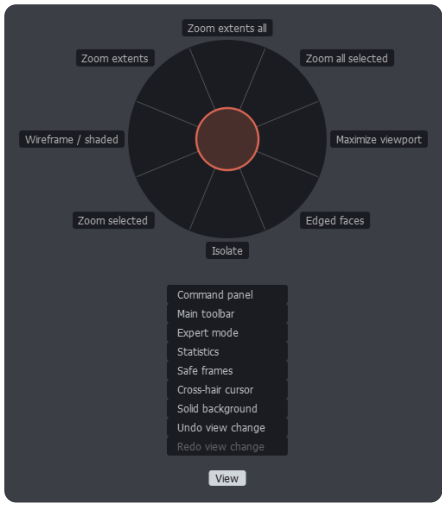
Changes with the situation (first match wins):

- **Show and hide** - when the object is an Editable Poly, sub-object level: object level
- **Poly: show and hide** - when the object is an Editable Poly

Ctrl+Alt+7 View

dial 15

NW Zoom extents	**N** Zoom extents all	**NE** Zoom all selected
W Wireframe / shaded	**View**	**E** Maximize viewport
SW Zoom selected	**S** Isolate	**SE** Edged faces

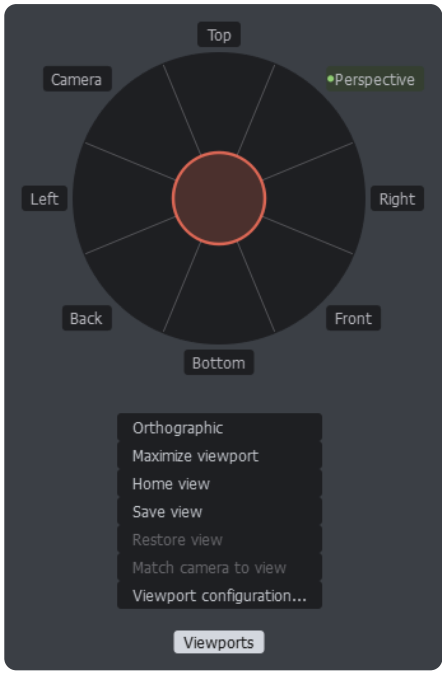


List under the dial: Command panel · Main toolbar · Expert mode · Statistics · Safe frames · Cross-hair cursor · Solid background · Undo view change · Redo view change

Ctrl+Alt+8 Viewports

dial 16

NW Camera	**N** Top	**NE** Perspective
W Left	Viewports	**E** Right
SW Back	**S** Bottom	**SE** Front

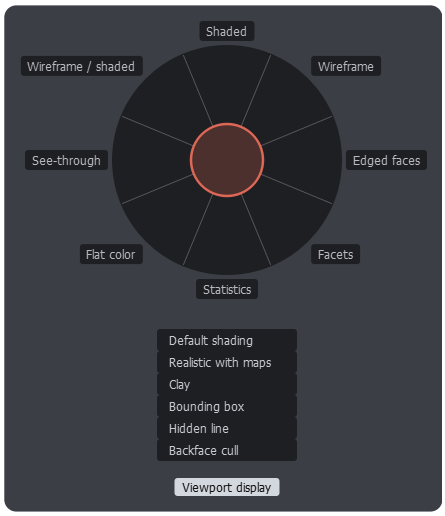


List under the dial: Orthographic · Maximize viewport · Home view · Save view · Restore view · Match camera to view · Viewport configuration...

Shift+Alt+1 Viewport display

dial 17

NW Wireframe / shaded	**N** Shaded	**NE** Wireframe
W See-through	Viewport display	**E** Edged faces
SW Flat color	**S** Statistics	**SE** Facets

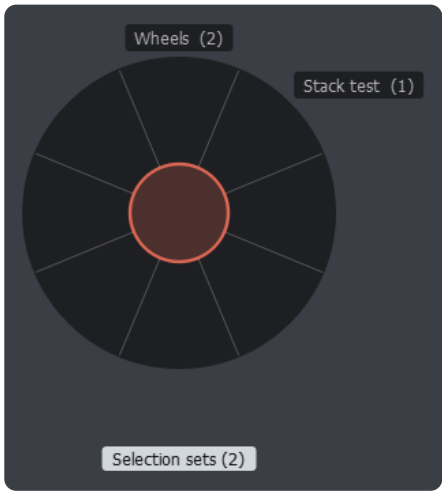


List under the dial: Default shading · Realistic with maps · Clay · Bounding box · Hidden line · Backface cull

Shift+Alt+2 Selection sets

dial 18

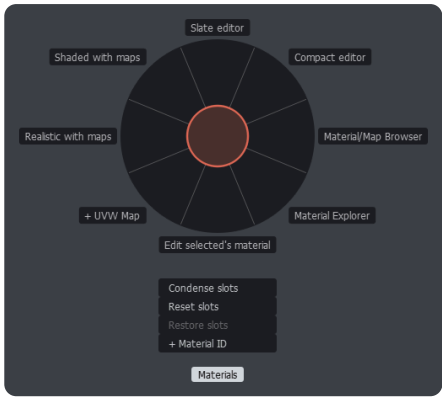
Filled live from the named selection sets of the scene.



Shift+Alt+3 Materials

dial 19

NW Shaded with maps	**N** Slate editor	**NE** Compact editor
W Realistic with maps	**Materials**	**E** Material/Map Browser
SW + UVW Map	**S** Edit selected's material	**SE** Material Explorer



List under the dial: Condense slots · Reset slots · Restore slots · + Material ID

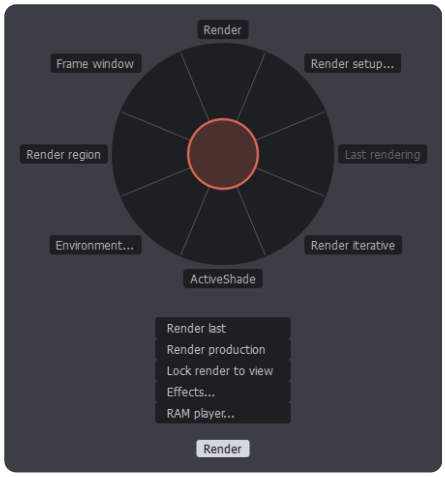
Changes with the situation (first match wins):

- **Materials (Slate)** - when cursor over the Slate Material Editor

Shift+Alt+4 Render

dial 20

NW Frame window	**N** Render	**NE** Render setup...
W Render region	**Render**	**E** Last rendering
SW Environment...	**S** ActiveShade	**SE** Render iterative

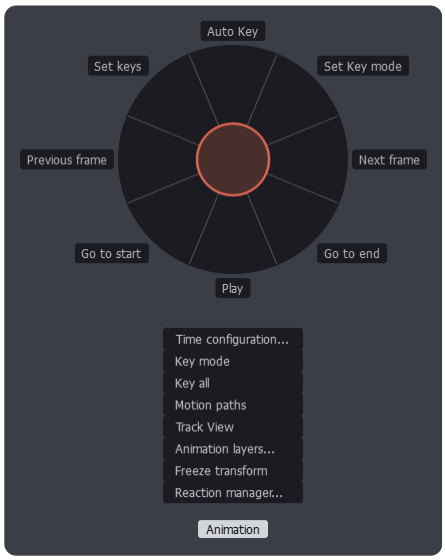


List under the dial: Render last · Render production · Lock render to view · Effects... · RAM player...

Shift+Alt+5 Animation

dial 21

NW Set keys	**N** Auto Key	**NE** Set Key mode
W Previous frame	**Animation**	**E** Next frame
SW Go to start	**S** Play	**SE** Go to end



List under the dial: Time configuration... · Key mode · Key all · Motion paths · Track View · Animation layers... · Freeze transform · Reaction manager...

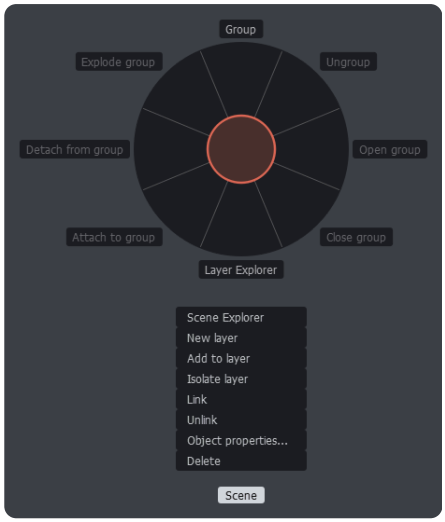
Changes with the situation (first match wins):

- **Track View** - when cursor over Track View

Shift+Alt+6 Scene

dial 22

NW Explode group	**N** Group	**NE** Ungroup
W Detach from group	Scene	**E** Open group
SW Attach to group	**S** Layer Explorer	**SE** Close group



List under the dial: Scene Explorer · New layer · Add to layer · Isolate layer · Link · Unlink · Object properties... · Delete

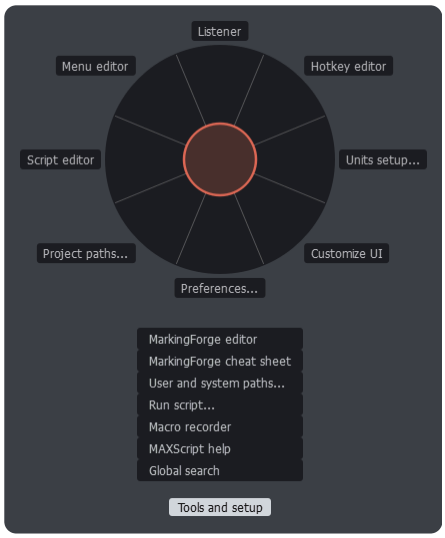
Changes with the situation (first match wins):

- **Scene Explorer** - when cursor over the Scene Explorer

Shift+Alt+7 Tools and setup

dial 23

NW Menu editor	**N** Listener	**NE** Hotkey editor
W Script editor	Tools and setup	**E** Units setup...
SW Project paths...	**S** Preferences...	**SE** Customize UI



List under the dial: MarkingForge editor · MarkingForge cheat sheet · User and system paths... · Run script... · Macro recorder · MAXScript help · Global search

Shift+Alt+8 File

dial 24

NW Hold	**N** Save	**NE** Save as...
W Merge...	File	**E** Save incremental
SW Import...	**S** Open...	**SE** Save copy as...



List under the dial: Fetch · Export... · Export selected... · Save selected... · Archive... · Summary info... · Reset

The dial library: more dials, one file each

Dial library... in the editor (1.1.0) holds 51 dials: the 24 above as shipped, 12 extra dials for one job each, and the 15 dials of the preset packs. Put one on a dial or add it as a new set.

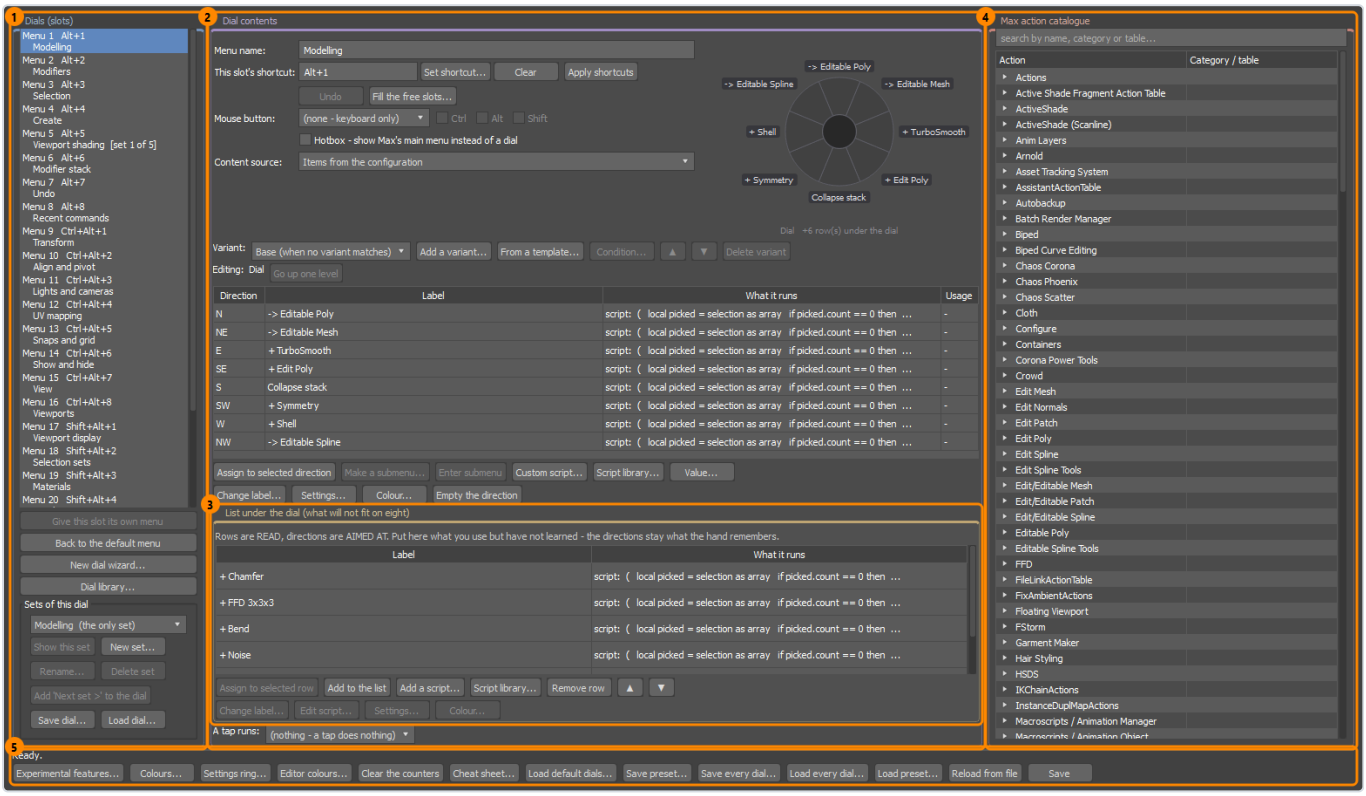
Extra dial	What it is for
Archviz: Build	Architecture objects in 3ds Max's own creation modes: walls, doors, windows, stairs, railings and plants - drag them in the viewport as from the Create panel.
Look-dev: Quick look	Checking a model at a glance: grey clay, no material, random wire colours, see-through, box display and the UV checker.
Menus: Hotbox - Max's menu	Max's whole main menu as tiles - the last page of Alt+5 (one wheel notch up; Alt+2 until 1.5.20), and until 1.4.4 the dial on its key itself. Put it on any key to have it there again.
Modelling: Clone and instance	Every way to make more of an object: clone, array, spacing along a path, snapshot, mirror - and the instance tools on the other side.
Modelling: Mesh cleanup	Find and fix what breaks smoothing and export: n-gons, triangles and holes on one side, the fixes on the other; reset the transform; reduce.
Modelling: Pivot and placement	Where an object's pivot is and where the object stands: pivot to the bottom or the centre, onto the ground, copy a transform from one object to another, align.
Modelling: Retopology	Rebuilding topology on an Editable Poly: loops added and removed, cuts, welds, and Quadify or ProOptimizer for a quick start.
Modelling: Smoothing and subdivision	Subdivision modifiers north and east, smoothing fixes south and west; switch every smoothing modifier of the selection off and on in one move.
Rigging: Links and helpers	Building a hierarchy: link and unlink, walk up and down it, freeze the transform, helpers to link to, and the deformers a rig starts with.
Viewport: Viewport display	Page 3 of Alt+5: grid, safe frames, statistics, ViewCube, selection display, isolate - and hiding whole categories in the list.
Viewport: Viewport lighting	Page 2 of Alt+5: shadows, ambient occlusion, highlights, scene or default lights, and materials with or without maps.
Viewport: Viewport views	Page 4 of Alt+5: top, front, left, right, perspective, orthographic, camera, maximize - zooms, layouts and the field of view in the list.

Pack dial	Where the pack puts it
Animation: Animate transform	From the Animation preset pack, where it sits on dial 9 (Ctrl+Alt+1).
Animation: Curves and layers	From the Animation preset pack, where it sits on dial 21 (Shift+Alt+5).
Animation: Keys	From the Animation preset pack, where it sits on dial 16 (Ctrl+Alt+8).
Animation: Playback and time	From the Animation preset pack, where it sits on dial 20 (Shift+Alt+4).
Archviz: Layers and groups	From the Archviz preset pack, where it sits on dial 22 (Shift+Alt+6).
Archviz: Place and array	From the Archviz preset pack, where it sits on dial 9 (Ctrl+Alt+1).
Archviz: References and files	From the Archviz preset pack, where it sits on dial 24 (Shift+Alt+8).
Archviz: Snaps	From the Archviz preset pack, where it sits on dial 13 (Ctrl+Alt+5).
LookDev: Cameras	From the LookDev preset pack, where it sits on dial 11 (Ctrl+Alt+3).
LookDev: Lights	From the LookDev preset pack, where it sits on dial 23 (Shift+Alt+7).
LookDev: Shading	From the LookDev preset pack, where it sits on dial 17 (Shift+Alt+1).
Modelling: Combine and cut	From the Modelling preset pack, where it sits on dial 17 (Shift+Alt+1).

Modelling: Poly tools	From the Modelling preset pack, where it sits on dial 14 (Ctrl+Alt+6).
Modelling: Symmetry and mirror	From the Modelling preset pack, where it sits on dial 16 (Ctrl+Alt+8).
Modelling: Viewport check	From the Modelling preset pack, where it sits on dial 20 (Shift+Alt+4).

5. The menu editor, control by control

MarkingForge > Menu editor... Changes are kept in the editor until you press **Save**; the plugin then reloads within about 150 ms, without restarting 3ds Max. Four panes, each with its own colour bar along the top edge: **Dials (slots)**, **Dial contents** with the **List under the dial** inside it, and the **Max action catalogue**; the bottom row holds the commands for the whole file.



The menu editor: the dials (1), the dial's contents (2), the list under the dial (3), the catalogue (4), the bottom row (5).

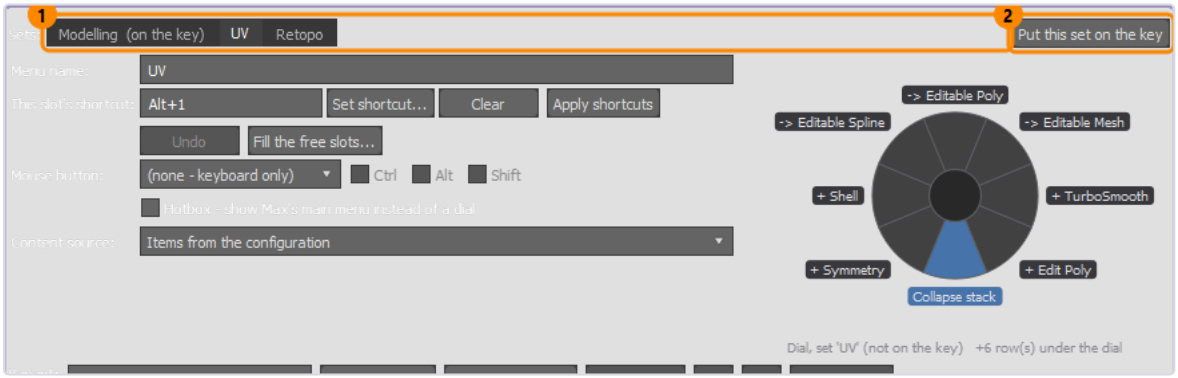
Left - the dials

Control	What it does
Dials (slots)	The 24 slots, each as *Menu N*, its shortcut (or *(no shortcut)*) and the menu name; *[inherited]* marks a slot that shows the default menu, *[set 2 of 3]* which set the key opens. Pick one to edit it.
Right-click a dial	In the list of dials or on the drawing of the dial (1.4.4): **Save Menu N (name) to a file...**, **Load a dial file into Menu N...**, **Put a dial from the library on Menu N...**, then **Save every dial...** and **Load every dial...** - the same commands as the buttons, where the dial is.
Give this slot its own menu	Makes an independent copy of the default menu for this slot. Needed before editing a slot that shows the default.
Back to the default menu	Deletes this slot's own menu - the slot shows the default again. What it held is lost, and so are the slot's sets.
New dial wizard...	Three pages. *1. Which dial, and its name* - with **Add it as a new set of this dial - the key keeps what it opens now** (the default) or **Replace what this dial shows**; *2. Eight commands* - click a direction, then double-click a command in the list on the right (or select it and press **Put into ...**; the next free direction comes next), or type the first letters of a command in a direction and pick it from the drop-down - a name typed in full is found too (1.5.55); *3. Ready* - a summary. A new set's tab opens; the key keeps opening what it opened.
Dial library...	Every dial MarkingForge ships, one file each (1.1.0), in three groups: *Built-in dials, as shipped* (24), *Extra dials* (12) and *From the preset packs* (15). The dial is drawn beside the list. **Put on Menu N** replaces what the dial shows (its

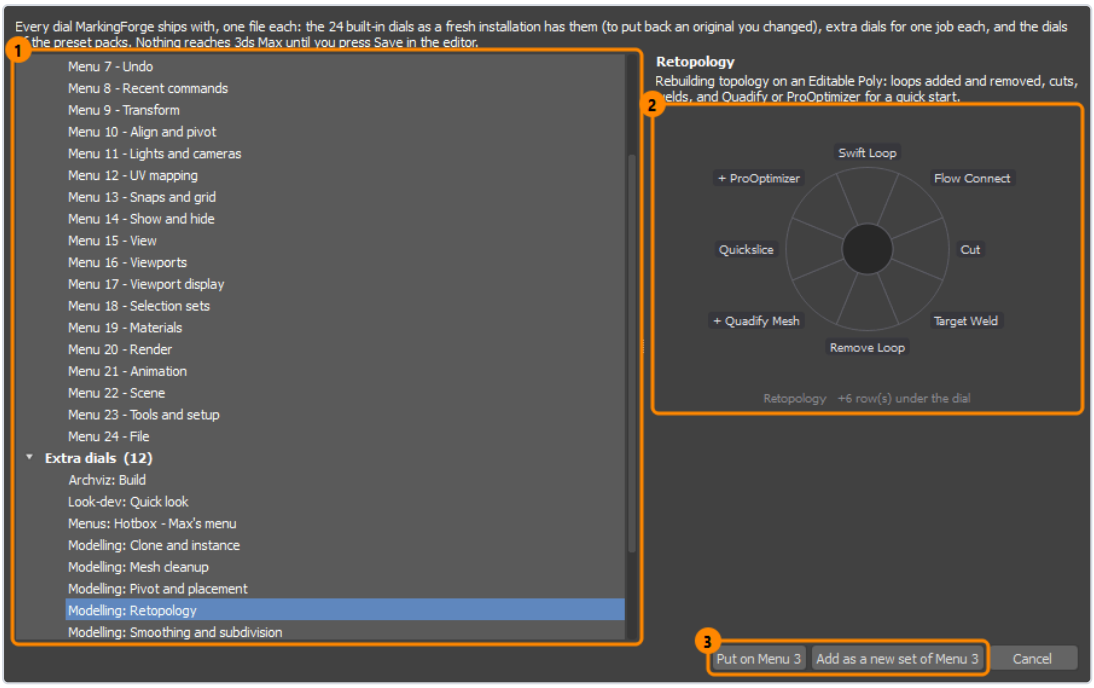
	key and mouse button stay) - it reads **Restore Menu N** when the original of this very dial is selected, which happens when the window opens; **Add as a new set of Menu N** keeps what the key opens. Nothing reaches 3ds Max until Save.
Sets of this dial	A list of the dial's sets - the one the key opens is marked - *shown now*; a dial without sets shows *(the only set)*. The list follows the open tab (1.4.3), so the buttons act on the set you are looking at. **Show this set** puts the chosen set on the key. **New set...** asks for a name, then **A copy of what the dial shows now** or **An empty dial**; the new set is the dial's next page and its tab opens - the key keeps opening what it opened (1.2.1). **Rename...**; **Delete set** (not the one on the key).
Add 'Next set >' to the dial	Puts a *Next set* > row on the list under the dial in every set and its variants - picking it puts the dial's next set on the key. Needs two sets or more.
Save dial... / Load dial...	One dial in a `.mfdial` file. Save writes the set chosen in the list. Load asks **A new set (keeps what the dial shows now)** or **Replace what the dial shows**; a file whose items run MAXScript asks first: **Remove the scripts** (the default) or **Keep them - I trust this file**.

Over the dial contents - the set tabs

Control	What it does
Sets: (tabs)	Shown once a dial has two sets or more (1.2.0): one tab per set, the one the key opens marked *(on the key)*. Click a tab to see and edit that set - every control below then works on it. The key opens the set on the key until the mouse wheel turns the dial to another one - from then on the page turned to last (1.4.8).
Put this set on the key	Makes the set of the open tab the one the key opens. The set it opened until now stays, one tab away.



The sets as tabs (1): another set open for editing, and the button that puts it on the key (2).



Dial library...: the built-in dials as shipped, the extra dials and the packs' dials (1), the chosen dial (2), put it on the dial or add it as a set (3).

Middle - the dial

Control	What it does
Menu name	The name shown in the dial's footer, in the cheat sheet and next to the slot.
The drawing of the dial	Beside the form: the dial being edited, north at the top. Click a wedge to select that direction; double-click a submenu (>) to step inside. Red = a command this 3ds Max lacks. Right-click it for the dial's file commands (above). Hidden while the middle pane is too narrow for it.
This slot's shortcut	Shows the key. **Set shortcut...** opens a window: press the keys there (Ctrl, Alt, Shift in any combination) and click **Set** (**Try again** for another combination). While it is open the dials and 3ds Max's own shortcuts are paused, so a key that already opens a dial can be recorded; it then moves from that dial to this one. The window also says when 3ds Max uses the key for another command. **Clear** (or Backspace) removes the key. Nothing reaches 3ds Max until **Apply shortcuts**.
Apply shortcuts	Writes the shortcuts of all 24 slots into 3ds Max at once. Shortcuts of other commands are kept. A backup is made first.
Undo (next to it)	Restores the shortcuts from before the last Apply.
Fill the free slots...	Offers Alt+1..8 / Ctrl+Alt+1..8 / Shift+Alt+1..8 for every slot without a shortcut and lists keys already used by something else before writing.
Mouse button (+ Ctrl / Alt / Shift)	Opens the dial by holding a mouse button with a modifier. A modifier is required (chapter 9). A button opens ONE dial, so on a slot that still shows the default menu the editor first offers to give the slot its own menu.
Hotbox - show Max's main menu instead of a dial	Turns the slot into a hotbox of 3ds Max's main menu, including entries other plugins add; the directions are then not used.
Content source	Items from the configuration, or a live source (chapter 7); the line under it says what the chosen source shows.
Variant	Choose which contents you edit: *Base* or one of the variants.
Add a variant...	New contents with a condition - starts as a copy of what you see.

From a template...	A ready-made variant: any context variant of the built-in dials (Editable Poly and Edit Poly by level, splines, cameras, lights, windows) with its condition, or a whole built-in dial (you give the condition). Added at the end of the list.
Condition...	Edits the selected variant's condition (chapter 6).
▲ / ▼	Moves the variant up or down. The first match wins - keep specific conditions above general ones.
Delete variant	Removes the variant and its contents; the base stays.
Go up one level	Leaves a submenu you are editing.
Directions table	N ... NW with label, what runs and the usage count. Fill a row by dragging a command from the catalogue, double-clicking a command, or **Assign to selected direction** (the first button under the table). Double-click a submenu row to step inside.
Make a submenu...	Turns the selected direction into a submenu (another dial) and steps into it.
Enter submenu	Edits the submenu under the selected direction.
Custom script... / Edit script...	Puts your own MAXScript under the direction, or edits it when the direction already runs a script. It runs with full rights (chapter 8).
Script library...	Puts a ready-made, tested script under the direction (chapter 8).
Value...	Turns the direction into a value slider (chapter 8).
Change label...	Your own caption instead of 3ds Max's name. Shorter is better on the dial.
Settings...	Whether an ordinary release opens this command's caddy: as the global setting, always, or never. Crossing the rim still reverses it. For a script: its second version *with parameters* (the command with its caddy, say) - one runs on an ordinary release, the other past the rim.
Colour...	This one item's caption and tile colour.
Empty the direction	Removes the item; the direction cancels.
Usage conclusions	Hints drawn from the usage counters, such as a direction you aim at often that is empty. Quick gestures and held dials are both counted. The editor never rearranges anything by itself.
A tap runs	What a press-and-release without movement runs: nothing, or one of the directions.
List under the dial	Up to twelve rows. The first row of buttons fills the list: **Assign to selected row** (the command selected in the catalogue in place of the selected row), **Add to the list** (the command selected in the catalogue), **Add a script...**, **Script library...**, **Remove row**, ▲ ▼ to reorder; commands can also be dragged in. The second row edits the selected row like a direction: **Change label...**, **Edit script...**, **Settings...**, **Colour...** (the caption only - a row has no tile).

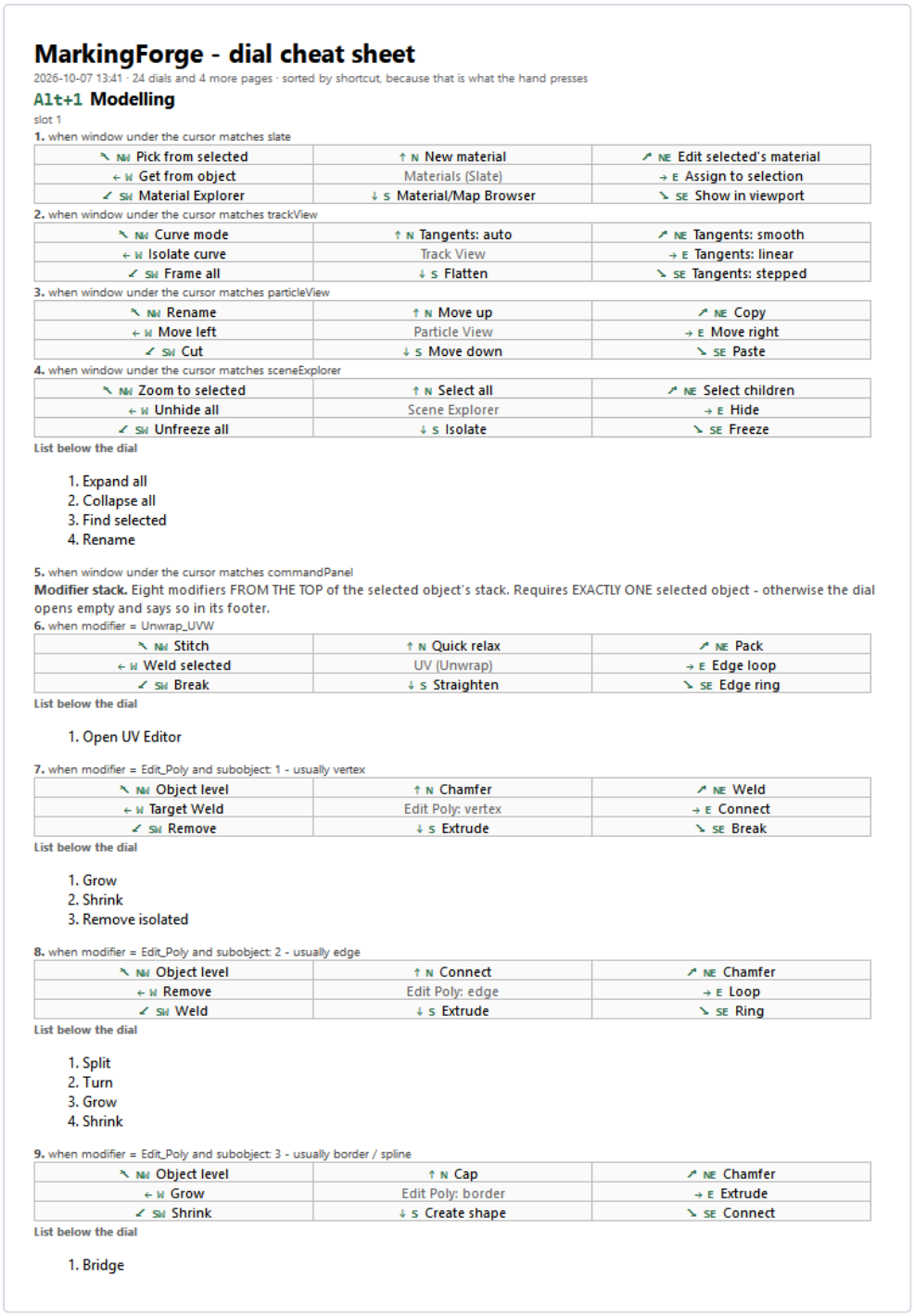
Right - the catalogue

Control	What it does
Search field	Filters 3ds Max's 4000+ actions and macroscripts by name, category or table. The same name can exist in several tables - the category column tells them apart.
Hover a command	Shows its table and id - useful when writing menus.json by hand.

Bottom row

Button	What it does
Experimental features...	The toggles of chapter 11, each with its reason.

Colours...	The colour editor (chapter 10).
Settings ring...	How far the dotted settings ring lies past the farthest caption (1.5.10) - see below.
Editor colours...	The colours of the editor window itself - panes, tables and lists, every column, text (chapter 10).
Clear the counters	Resets the usage statistics. Menus are not changed.
Cheat sheet...	Writes your SAVED dials - with their current keys and mouse bindings, list rows and submenu paths - into one HTML and PDF document to print, sorted by shortcut, and lists what needs checking (dials that would open empty, clashing mouse bindings). Unsaved edits are not included; a warning says so.
Load default dials...	Brings in the built-in 24 dials. By default only slots without their own menu are filled; tick the option to replace the others too - every dial then also gets its default pages and the default dial colours back (1.5.28). Shows the list before changing.
Save preset...	Saves your dial layout to a file you can share - every dial, its pages, the dial colours and the default menu (1.5.28); without usage data, paths or the editor's own colours.
Save every dial...	Asks for a folder and makes a new one in it, *MarkingForge dials YYYY-MM-DD HH-MM*: each of the 24 dials, and every other set of a dial, in its own `.mfdial` file named *NN name.mfdial*, with a list (`MarkingForge dials.json`, 1.4.4) of which file is which (chapter 14).
Load every dial...	Puts back EXACTLY what such a folder saved (1.5.55): every dial, its other sets in their order, the dials that showed the default menu, the default menu itself, the mouse buttons, the palette of every dial and every option - changes made since are dropped. Choosing the folder you saved INTO loads its newest *MarkingForge dials* folder. A folder saved before 1.5.55 puts back the dials and the palette only. Scripts ask first (**Remove the scripts** / **Keep them - I trust these files**), then the editor says how many dials and sets it will load. A dial keeps its own mouse button; dials the folder does not hold stay as they are. A folder without the list (saved before 1.4.4) is read from the file names - the first file of a dial is the one it showed. A damaged file refuses the whole load. Nothing is written until Save. Also on the right-click menu of a dial.
Load preset...	Loads a layout, replacing the slots it contains and their pages; a preset of the whole layout also sets the dial colours as saved, a pack of a few dials leaves them alone. Script and value items are removed by default wherever they sit - directions, list rows, the tap item, submenus and variants - because a preset is somebody else's code; the editor shows the contents first and reports how many dials it loaded.
Reload from file	Discards unsaved changes and reads menus.json again.
Save	Writes menus.json; the plugin reloads it by itself. If menus.json changed on disk after the editor loaded it (another editor, a tick in the Experimental features menu, a page switched from a dial), Save asks (1.5.55): **Overwrite with my edits**, **Save a copy...** (by default `menus.json.conflict.json`) or **Cancel** - nothing is overwritten unless you say so. A file that could not be read must be repaired and reloaded before Save works.
Closing the window	Shortcuts set but not applied are offered first (*Apply them now?* Yes, No, Cancel). With unsaved changes, Esc or the X asks: Save, Discard or Cancel. Nothing is lost silently. The editor's own colours are not part of this - they are saved when you press OK in **Editor colours...**.



MarkingForge - dial cheat sheet

2026-10-07 13:41 · 24 dials and 4 more pages · sorted by shortcut, because that is what the hand presses

Alt+1 Modelling

slot 1

1. when window under the cursor matches slate

↖ NW Pick from selected	↑ N New material	↗ NE Edit selected's material
← W Get from object	Materials (Slate)	→ E Assign to selection
↙ SW Material Explorer	↓ S Material/Map Browser	↘ SE Show in viewport

2. when window under the cursor matches trackView

↖ NW Curve mode	↑ N Tangents: auto	↗ NE Tangents: smooth
← W Isolate curve	Track View	→ E Tangents: linear
↙ SW Frame all	↓ S Flatten	↘ SE Tangents: stepped

3. when window under the cursor matches particleView

↖ NW Rename	↑ N Move up	↗ NE Copy
← W Move left	Particle View	→ E Move right
↙ SW Cut	↓ S Move down	↘ SE Paste

4. when window under the cursor matches sceneExplorer

↖ NW Zoom to selected	↑ N Select all	↗ NE Select children
← W Unhide all	Scene Explorer	→ E Hide
↙ SW Unfreeze all	↓ S Isolate	↘ SE Freeze

List below the dial

1. Expand all
2. Collapse all
3. Find selected
4. Rename

5. when window under the cursor matches commandPanel

Modifier stack. Eight modifiers FROM THE TOP of the selected object's stack. Requires EXACTLY ONE selected object - otherwise the dial opens empty and says so in its footer.

6. when modifier = Unwrap_UVW

↖ NW Stitch	↑ N Quick relax	↗ NE Pack
← W Weld selected	UV (Unwrap)	→ E Edge loop
↙ SW Break	↓ S Straighten	↘ SE Edge ring

List below the dial

1. Open UV Editor

7. when modifier = Edit_Poly and subobject: 1 - usually vertex

↖ NW Object level	↑ N Chamfer	↗ NE Weld
← W Target Weld	Edit Poly: vertex	→ E Connect
↙ SW Remove	↓ S Extrude	↘ SE Break

List below the dial

1. Grow
2. Shrink
3. Remove isolated

8. when modifier = Edit_Poly and subobject: 2 - usually edge

↖ NW Object level	↑ N Connect	↗ NE Chamfer
← W Remove	Edit Poly: edge	→ E Loop
↙ SW Weld	↓ S Extrude	↘ SE Ring

List below the dial

1. Split
2. Turn
3. Grow
4. Shrink

9. when modifier = Edit_Poly and subobject: 3 - usually border / spline

↖ NW Object level	↑ N Cap	↗ NE Chamfer
← W Grow	Edit Poly: border	→ E Extrude
↙ SW Shrink	↓ S Create shape	↘ SE Connect

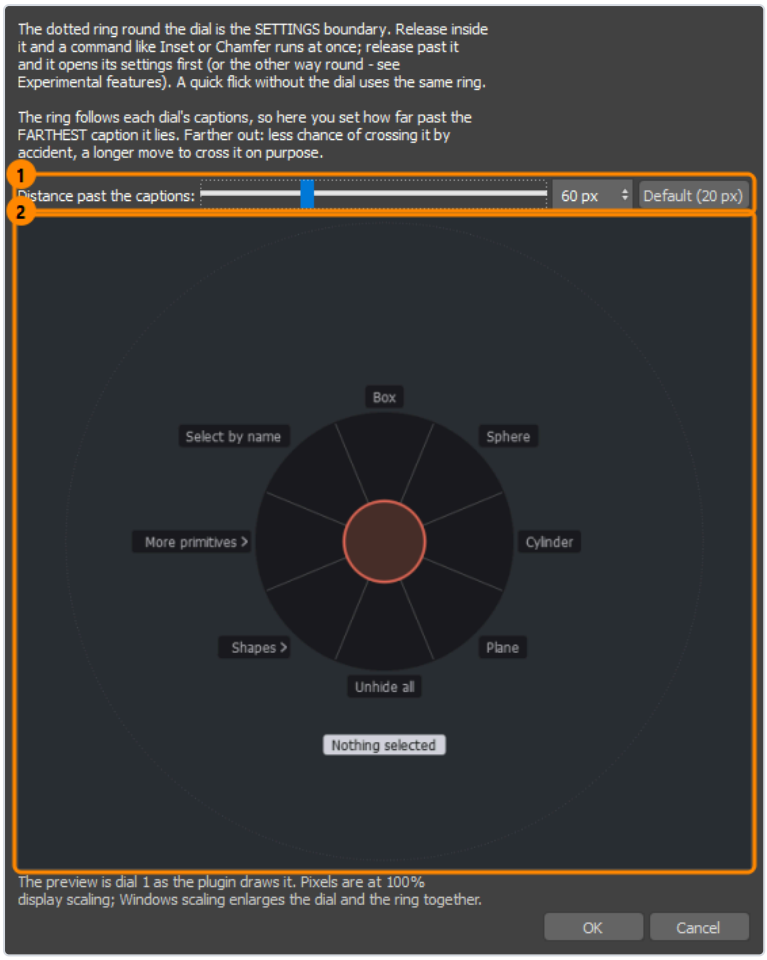
List below the dial

1. Bridge

The first page of the cheat sheet (Cheat sheet...): every dial as a 3x3 square, sorted by shortcut, a dial with pages with every page - print it and keep it beside the keyboard for the first week.

Settings ring...

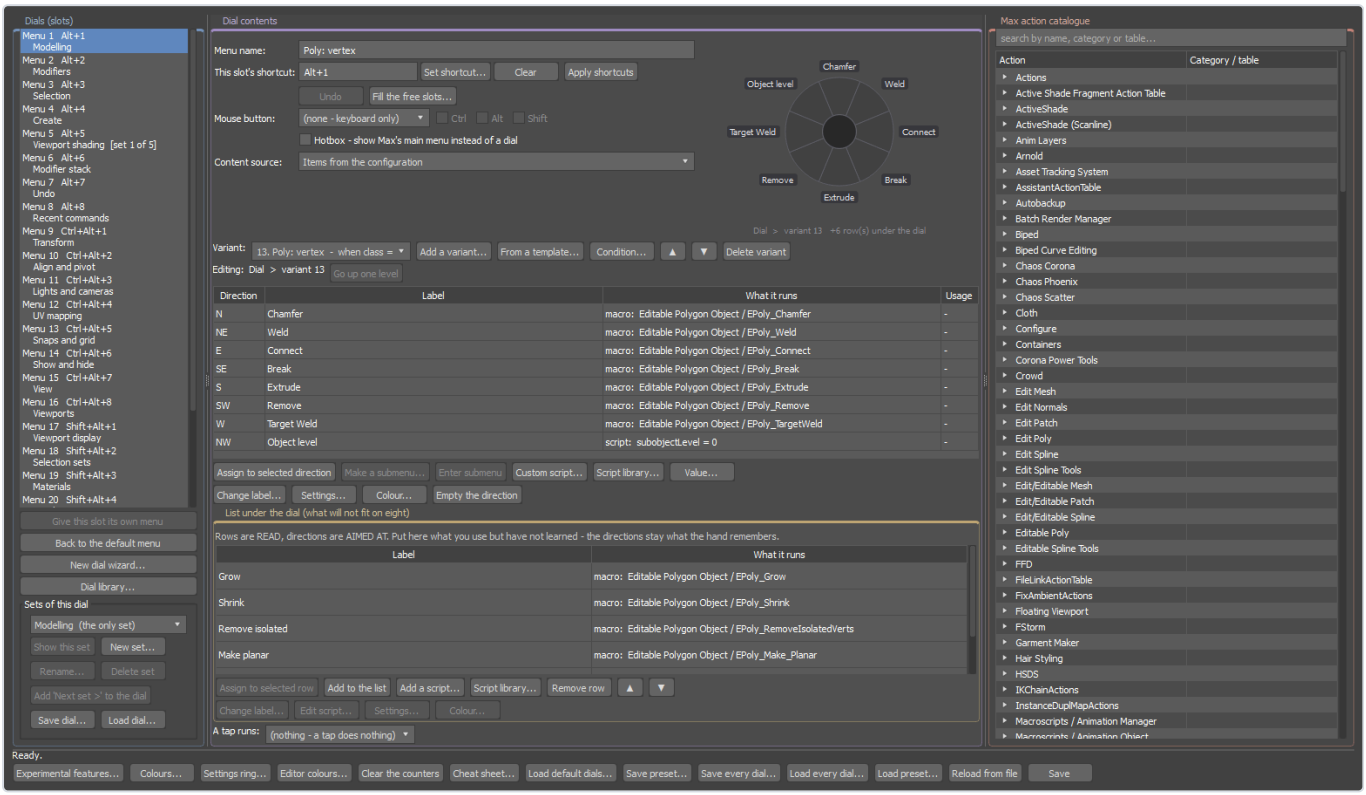
The dotted circle round the dial is the settings boundary (chapter 2): released inside it, a command with a caddy does its ordinary thing, released past it the opposite - and a flick is measured against the same circle. Its radius follows each level's captions, so what you set is the **distance past the farthest caption**: 0 to 200 px, 20 px by default (1.5.10). Farther out means fewer caddies skipped by accident and a longer move to skip one on purpose; nearer, the other way round. The picture in the window is the real dial, drawn by the plugin with the number you are choosing (it needs plugin 1.5.10 in this 3ds Max). OK puts the number into the editor; **Save** hands it to the plugin. One setting for every dial, stored as `rimDistance`. The dial's window always has room for the ring at the distance you set, so the window's edge never pulls it in (1.5.13: up to 1.5.12 a dial with a full-width side caption kept the ring on that caption up to 20 px).



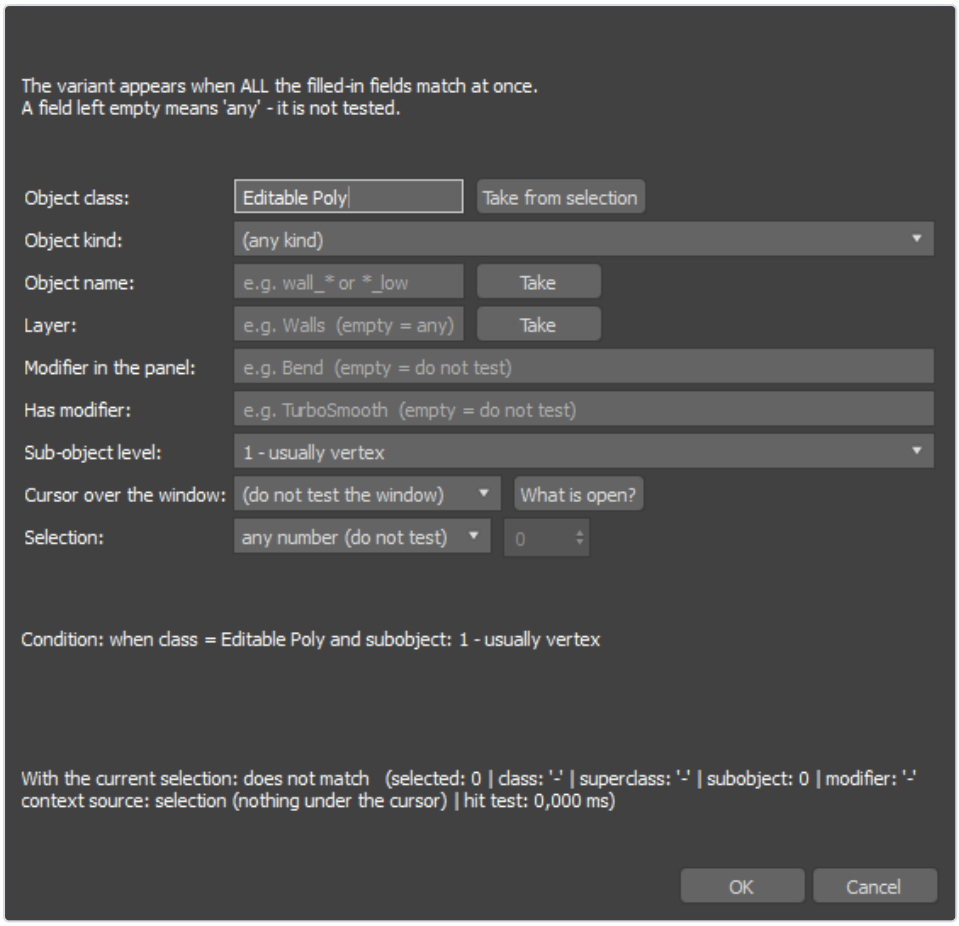
Settings ring...: the distance (1) and the real dial with the ring where it will be (2).

6. Contextual variants (conditions)

A variant appears when **all** filled fields match; an empty field is not tested. Variants are tried from the top and the first match wins, so put the specific ones first - for example *Edit Poly at Vertex level* above *Edit Poly*.



The editor with the variant *Poly: vertex* of the Modelling dial selected - what Alt+1 shows on an Editable Poly at vertex level.



Condition... - the fields of a variant's condition. **Take from selection** fills them from the object selected in 3ds Max.

Field	Matches when	Tips
Object class	The selected object's base object is of this class, for example `Editable Poly`.	**Take from selection** fills it in the exact spelling. Class looks UNDER the modifiers: an Editable Poly with a TurboSmooth on top is still an Editable Poly. Both spellings (`Editable Poly` / `Editable_Poly`) work.
Object kind	The object is of this kind: geometry, shape, light, camera, helper.	Wider than a class - one variant for every light.
Modifier in the panel	The Modify panel is currently editing this modifier, for example `Edit_Poly`.	Put these ABOVE class variants: an Edit Poly on an Editable Poly matches both.
Has modifier	The object has this modifier anywhere in its stack.	Answers "does it have it", not "is it open".
Sub-object level	object level (0); 1 - usually vertex; 2 - usually edge; 3 - usually border / spline; 4 - usually polygon; 5 - usually element.	The numbers mean vertex/edge/border/polygon/element on a poly and vertex/segment/spline on a shape.
Cursor over the window	The cursor is over that window (table below).	**What is open?** lists the windows open right now with the word to type.
Selection	nothing selected; exactly one object; two or more; at least one, or an exact number.	*nothing selected* gives an empty scene its own menu.
Object name	The object's name matches a pattern with `*`, for example `Rig_*`.	Handy for rigs and props named by convention. **Take** fills in the selected object's name. Without `*` the whole name must match (field `name` in menus.json).

Layer	The object is on this layer.	The whole name, no `*`. **Take** fills in the selected object's layer (field `layer`).

Windows the plugin recognises

Word	Window	Details
(empty)	(do not test the window)	The rule ignores where the cursor is.
`viewport`	3D view	Decided by Max's SDK, not by a window name - so it works regardless of window layout and of language.
`commandPanel`	Command panel	The right-hand panel with the Create / Modify / Hierarchy tabs.
`slate`	Material editor (Slate)	The Slate Material Editor, together with its child windows.
`materialBrowser`	Material/Map Browser	The material and map list inside Slate. NARROWER than 'slate', which also covers it - use this one when the rule is about the browser itself.
`materialParams`	Material parameters (Slate)	The parameter panel of the material under the cursor, inside Slate. Also narrower than 'slate'.
`uvEditor`	UV editor (Edit UVWs)	The Unwrap UVW modifier's editor window - its UV canvas, side panels and bottom toolbars. MEASURED: an old Windows dialog of the generic class '#32770', known by its canvas, a child window of class 'UnwrapView'. NOT the same as 'Modifier in the panel: Unwrap_UVW', which holds wherever the cursor is while the modifier is open.
`trackView`	Track View	The curve and key editor. Recognised by object name, because its class is shared by many docked windows. MEASURED: the object is called 'TrackView - Track View - Curve Editor', the class is MaxSDK::QmaxDockWidget.
`sceneExplorer`	Scene Explorer	The scene object list. The same generic class as Track View, so it is likewise decided by object name: 'Scene Explorer - <name>'.
`particleView`	Particle View	The ParticleFlow window. The ONLY one on this list that is not a Qt window at all but an old Windows dialog of class 'Particle View Dialog'. On the Qt side it belongs to the main window, so without this word it could not be told apart from it.
`ribbon`	Ribbon (modelling toolbar)	The ribbon with the modelling tools. MEASURED: object name 'Ribbon', class MaxSDK::QmaxDockWidget - the generic docked class, so the name decides.
`timeSlider`	Time slider	The animation bar under the views. MEASURED: object name 'TimeSlider', class QmaxTimeSliderDockWidget.
`floatingViewport`	Floating view	A view torn off into a window of its own. NOT the same as '3D view': that one is decided by the SDK against the main window's views, while a torn-off panel is a separate window.
`mainWindow`	Max's main window	The main window's OWN surface - toolbars, time slider, background. It does NOT cover what Max hosts inside it: neither the 3D view nor old dialogs such as Particle View. Until 0.39.1 it did, and that was wrong: the 'main window' rule fired over half of Max.

With the experimental *Menu under the cursor* on (default), the object UNDER the cursor decides the variant when there is one; otherwise the selection does. Picking an item then selects that object and runs the command on it in one undo. Hotbox entries and type-to-search results run without changing the selection.

7. Content sources

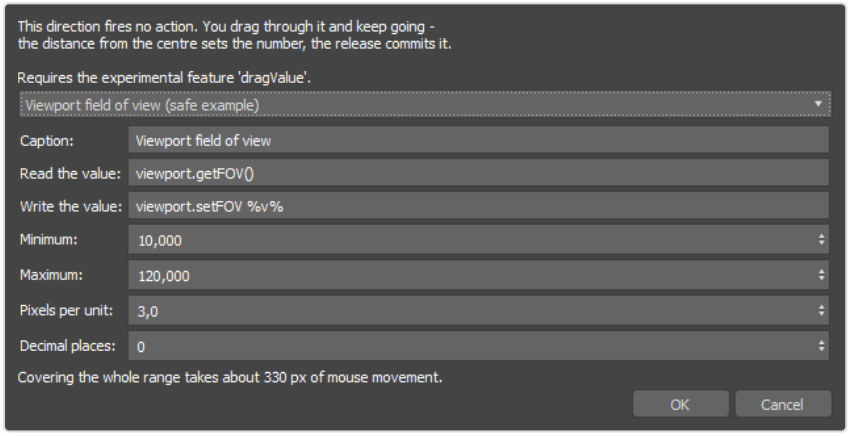
Source	Word in menus.json	What the dial shows
Items from the configuration	`config`	Eight directions exactly as you set them here. An ordinary dial.
Modifier stack	`modifiers`	Eight modifiers FROM THE TOP of the selected object's stack. Requires EXACTLY ONE selected object - otherwise the dial opens empty and says so in its footer.
Undo stack	`undo`	The last eight operations BY NAME. Direction N undoes one, NE two, and so on. Requires the experimental feature 'undoDial'.
Recent commands	`recent`	The eight most recently RUN commands, newest first - from any dial, a tap or the search. For REPEATING, as opposed to the undo stack. The history lives in memory and starts afresh when Max starts. Requires the 'recentDial' toggle.
Selection sets	`selectionSets`	The scene's named selection sets as eight directions. With more than eight it shows the first eight. Requires the toggle 'selectionSetsDial'.

A source can be set on the whole dial or on one variant only - for example the modifier stack only while the cursor is over the command panel (the built-in Modelling dial does exactly this). Items typed into a dial with a source are kept, not lost; they come back when you switch the source to *Items from the configuration*.

8. Item types

Type	Runs	Fields in menus.json
Action	A 3ds Max action, from any action table - built-in or another plugin's.	`table` (number), `id` (text), optional `label`, `settings`
Macro	A macroscript.	`category`, `name`, optional `label`, `settings`
Script	Your own MAXScript code, with full rights.	`code`, optional `label`, `codeSettings` (code for the settings variant of the gesture)
Submenu	Another dial, opened by moving through the direction. Up to three levels.	`label`, `items` (eight directions), optional `list`
Value	A slider: the distance from the centre sets a number, the release commits it. Needs *dragValue*.	`get` (expression returning the start value), `set` (code containing `%v%`), `min`, `max`, `pxPerUnit`, `decimals` (0-4)

The value slider in the editor



Value... with its first ready-made example (viewport field of view).

Field	Meaning
Caption	The dial caption; the number appears next to it.
Read the value	A MAXScript expression returning the starting number, read when the cursor enters the direction. No number (nothing selected) shows **?**.
Write the value	Code run on release. Must contain `%v%`, which becomes the value.
Minimum / Maximum	The range the gesture cannot leave.
Pixels per unit	Mouse movement per one unit. Smaller = more sensitive. For 0-4 use about 40, for 0-500 about 2.
Decimal places	0 = whole numbers.
Example	Ready-made examples; the first (viewport field of view) needs no selection.

```
{ "type": "value", "label": "FOV",
  "get": "viewport.getFOV()",
  "set": "viewport.setFOV %v%",
  "min": 10, "max": 120, "pxPerUnit": 3, "decimals": 0 }
```

Scripts: ready-made or your own

A script direction runs MAXScript with full rights. There are two ways to get one, and both put the same kind of item on the dial:

Button	What you get
Script library...	One of 102 ready-made scripts. Each was run in 3ds Max on a test scene before it went into the list. Pick one - it arrives with its caption.
Custom script...	Your own code - typed, pasted from the Listener or from a script you already use.
Edit script...	Shown instead of *Custom script...* when the direction already runs a script: opens its code. The list under the dial has the same three buttons.

The script library

Group	Caption	What it does
Objects and pivots	**Pivot to bottom**	Moves each selected object's pivot to the centre of its bottom face - so it sits on the grid and scales from the floor.
Objects and pivots	**Drop to the ground**	Moves each selected object straight down (or up) until its lowest point is at Z = 0.
Objects and pivots	**Centre pivot**	Puts each selected object's pivot in the centre of its bounding box.
Objects and pivots	**Reset XForm + collapse**	Bakes rotation and scale into the mesh and clears the stack - the usual clean-up before exporting or before modifiers that misbehave on a scaled object.
Objects and pivots	**Clone in place**	Makes an independent copy of the selection on the same spot and selects the copies - Shift-drag without the drag.
Objects and pivots	**Make unique**	Turns instances and references among the selection into independent copies.
Objects and pivots	**Pivot aligned to world**	Turns each selected object's pivot back to the world's axes, leaving the mesh where it is - after a rotation, before aligning or exporting.
Objects and pivots	**Pivot to world origin**	Puts each selected object's pivot at 0,0,0 - for scaling or rotating several objects around the same point.
Objects and pivots	**Centre on the origin**	Moves each selected object so the centre of its bottom face is at 0,0,0.
Objects and pivots	**Attach to the first**	Turns the first selected object into an Editable Poly and attaches every other selected geometry object to it - one object from many.
Objects and pivots	**Split into elements**	Detaches every element of each selected Editable Poly to an object of its own - the opposite of Attach.
Topology (Editable Poly)	**Select n-gons**	Selects every polygon with five or more sides on the selected Editable Poly - the quickest topology check before subdividing.
Topology (Editable Poly)	**Select triangles**	Selects every three-sided polygon on the selected Editable Poly.
Topology (Editable Poly)	**Select holes**	Selects the open edges (borders) of the selected Editable Poly - every hole in the mesh.
Topology (Editable Poly)	**Weld close vertices**	Welds vertices closer than 0.01 units on every selected Editable Poly - cleans up doubled vertices after an import or a mirror.

Topology (Editable Poly)	**Auto smooth 30**	Sets smoothing groups by a 30-degree angle on every selected Editable Poly.
Topology (Editable Poly)	**Select faces facing up**	Selects the polygons of the selected Editable Poly that face up (Z) - tops of walls, floors, ledges.
Topology (Editable Poly)	**Select faces facing down**	Selects the polygons of the selected Editable Poly that face down - undersides to delete or to leave unmapped.
Topology (Editable Poly)	**Cap holes**	Closes every hole of each selected Editable Poly with a polygon.
Topology (Editable Poly)	**Flip normals**	Turns every polygon of each selected object the other way round - for a mesh that renders inside out. Baked into an Editable Poly with no modifiers; a Normal modifier on anything else.
Topology (Editable Poly)	**Clear smoothing**	Takes every smoothing group off each selected Editable Poly - hard, faceted shading.
Modifiers	**Smoothing on/off**	Switches every TurboSmooth and MeshSmooth on the selected objects on or off - a fast low-poly / high-poly check.
Modifiers	**+ UVW Map (box)**	Adds a box-projection UVW Map to every selected object.
Modifiers	**Remove top modifier**	Deletes the topmost modifier of every selected object.
Modifiers	**Extrude (Edit Poly)**	For an Edit Poly modifier at polygon level: an ordinary release starts an interactive Extrude, past the rim opens the Extrude caddy instead. *(two versions - see below)*
Modifiers	**+ Shell**	Adds a Shell of 1 unit outwards to every selected object - thickness for a plane or an open mesh.
Modifiers	**+ Symmetry**	Adds a Symmetry modifier (mirrored across X) to every selected object - model one half, see both.
Modifiers	**+ Chamfer**	Adds a Chamfer modifier (0.5 units, 3 segments) to every selected object - softened edges that catch the light.
Modifiers	**+ Edit Poly**	Adds an Edit Poly modifier to every selected object - editing that keeps the stack below it.
Modifiers	**+ FFD 3x3x3**	Adds an FFD 3x3x3 lattice to every selected object - for bending the whole form by a few control points.
Materials and display	**Grey clay material**	Gives the selection one neutral grey material - for checking forms and lighting without textures.
Materials and display	**Remove material**	Clears the material from every selected object.
Materials and display	**Random wire colour**	Gives every selected object its own random wire colour - tells parts apart in a crowded scene.
Materials and display	**See-through on/off**	Switches See-Through display (Alt+X) for each selected object.
Materials and display	**Display as box on/off**	Shows each selected object as its bounding box, or back - lightens heavy objects in the viewport.
Materials and display	**UV checker material**	Gives the selection a material with an 8 x 8 checker, shown in the viewport - stretched or uneven squares show bad UVs.
Materials and display	**Material = wire colour**	Gives each selected object a material of its own in its wire colour - a quick colour-coded render.

Materials and display	**Backface cull on/off**	Hides or shows the back of each selected object's polygons in the viewport - makes flipped normals and inside walls visible.
Materials and display	**Random colour materials**	Gives every selected object a material of its own in a random colour - tells parts apart in a render.
Materials and display	**Merge same-named mats**	Makes every object whose material has the same name share ONE of them - the clean-up after merging or importing scenes.
Materials and display	**First's material to all**	Gives every selected object the material of the first one.
Materials and display	**Wire colour = material**	Sets each selected object's wire colour to its material's base colour - the viewport matches the render.
Selection and transforms	**Select same material**	Selects every object that has the same material as the selected one.
Selection and transforms	**Select instances**	Adds every instance of the selected objects to the selection.
Selection and transforms	**Copy transform**	Remembers the selected object's position, rotation and scale (pair it with Paste transform).
Selection and transforms	**Paste transform**	Gives every selected object the transform remembered by Copy transform.
Selection and transforms	**Reset rotation**	Removes the rotation of each selected object and keeps it where it is.
Selection and transforms	**Select same type**	Selects every visible object of the same kind as the selected one - all the boxes, all the lines, all the Editable Polys.
Selection and transforms	**Select children**	Adds every child of the selected objects to the selection, all the way down the hierarchy.
Selection and transforms	**Select without material**	Selects every visible geometry object that has no material - the ones that would render in their wire colour.
Selection and transforms	**Random rotation (Z)**	Turns each selected object by a random angle around Z - breaks up repeated copies (rocks, plants, props).
Selection and transforms	**Random scale (10 %)**	Scales each selected object by a random amount between 90 and 110 percent.
Selection and transforms	**Spread evenly along X**	Places the selected objects at equal steps between the leftmost and the rightmost one, along X.
Shapes and splines	**Renderable on/off**	Switches each selected shape's 'Enable in renderer' and 'Enable in viewport' together - a spline as a tube, or back.
Shapes and splines	**Close all splines**	Closes every open spline of each selected Line or Editable Spline.
Shapes and splines	**+ Extrude 10**	Adds an Extrude of 10 units (capped) to every selected shape - a profile into a solid.
Shapes and splines	**Spline from edges**	Makes a new spline object from the selected edges of the Editable Poly.
Shapes and splines	**Lighter splines (2)**	Sets 2 interpolation steps with Optimize on every selected shape - lighter meshes from renderable splines.
Shapes and splines	**+ Lathe**	Adds a Lathe modifier (360 degrees) to every selected shape - a profile into a vase, a column, a glass.

Shapes and splines	**Renderable, 2 thick**	Makes every selected shape renderable and visible in the viewport as a tube 2 units thick.
Scene and layers	**Selection to a new layer**	Makes a new layer (MF Layer 1, 2, ...) and moves the selection onto it.
Scene and layers	**Freeze selection**	Freezes the selected objects - they stay visible but cannot be selected or moved by accident.
Scene and layers	**Unfreeze all**	Unfreezes every object in the scene.
Scene and layers	**Zoom to selection**	Frames the selection in every viewport.
Scene and layers	**Size to the status bar**	Shows the size of the selection (X, Y, Z, in scene units) in the status bar for five seconds, and in the Listener.
Scene and layers	**Rename in sequence**	Renames the selection after the first object: Chair_01, Chair_02, ... (digits and underscores at the end of the first name are dropped).
Scene and layers	**Layers by object type**	Puts the selection - or the whole scene when nothing is selected - on layers by kind: Geometry, Shapes, Lights, Cameras, Helpers.
Scene and layers	**Select whole layer**	Selects every object on the layers of the selected objects.
Scene and layers	**Delete empty layers**	Deletes every layer that holds no objects (never layer 0 or the current one).
Scene and layers	**Scene report**	Counts the scene - geometry and its polygons, shapes, lights, cameras, materials - into the status bar and the Listener.
Modelling (advanced)	**Select hard edges (30)**	Selects every edge of the selected Editable Poly whose two polygons meet at more than 30 degrees - the edges to chamfer, crease or split UVs on.
Modelling (advanced)	**Select edge loop**	Extends the selected edges of the Editable Poly to their whole loops.
Modelling (advanced)	**Select edge ring**	Extends the selected edges of the Editable Poly to their rings - the parallel edges across a strip.
Modelling (advanced)	**Random 20 % of faces**	Selects a random fifth of the polygons of the selected Editable Poly - for greebles, damage, variation.
Modelling (advanced)	**Extrude faces (5)**	Extrudes the selected polygons of the Editable Poly by 5 units, as a group.
Modelling (advanced)	**Bevel faces**	Bevels the selected polygons of the Editable Poly: 3 units out, 1 unit narrower - a quick panel or button.
Modelling (advanced)	**Detach faces to object**	Detaches the selected polygons of the Editable Poly into a new object of their own.
Modelling (advanced)	**Make faces planar**	Flattens the selected polygons of the Editable Poly onto one plane - for panels that came out warped.
Modelling (advanced)	**Bake TurboSmooth (1)**	Adds TurboSmooth (1 iteration) to each selected object and collapses its stack - a smoothed, editable mesh.
Modelling (advanced)	**Symmetrize X (baked)**	Mirrors each selected object across its pivot on its own X with a Symmetry modifier and collapses the stack to an Editable Poly.
Modelling (advanced)	**Greeble: random extrude**	Extrudes about a third of the polygons of the selected Editable Poly, each by its own random height - instant sci-fi surface detail.
Modelling (advanced)	**Chamfer edges (0.5)**	Chamfers the selected edges of the Editable Poly: 0.5 units, 2 segments.

Modelling (advanced)	**Connect edges (2)**	Connects the selected edges of the Editable Poly with two new edge loops.
Modelling (advanced)	**Delete half (-X)**	Deletes the polygons of the selected Editable Poly that lie left of its pivot (negative X) - the start of a symmetrical model.
Modelling (advanced)	**Bridge borders**	Bridges the two selected border loops of the Editable Poly with polygons.
UV mapping	**UVW box (100 units)**	Adds a box UVW Map of 100 x 100 x 100 units to every selected object - one texture scale on every object.
UV mapping	**UVW cylindrical (capped)**	Adds a cylindrical UVW Map with capped ends to every selected object - for pipes, posts and bottles.
UV mapping	**Unwrap + flatten**	Adds Unwrap UVW to the one selected object and flattens its UVs into islands.
UV mapping	**Copy UVs to channel 2**	Copies map channel 1 of every selected object to channel 2 - the usual start of lightmap or AO UVs.
UV mapping	**Checker 2x finer**	Doubles the tiling of the checker of 'UV checker material' on the selection - for small UV islands.
Layout and placement	**Snap to grid (10)**	Moves each selected object to the nearest multiple of 10 units on X, Y and Z.
Layout and placement	**Arrange in a grid**	Lays the selection out in rows and columns from the first object, spaced by the largest of them.
Layout and placement	**Stack on top**	Stacks the selected objects on top of each other, each centred on the one before.
Layout and placement	**Align bottoms to first**	Moves every selected object up or down so its bottom is level with the first one's.
Layout and placement	**Drop onto surface**	Drops each selected object straight down onto whatever geometry is below it.
Layout and placement	**Spread first to last**	Places the selected objects at equal steps on the straight line from the first to the last one, in 3D.
Layout and placement	**Random position (5)**	Moves each selected object by a random amount, up to 5 units on X and Y - breaks up a too-even layout.
Layout and placement	**Scatter onto surface**	Select the object to copy, then the surface: 20 instances of the first land at random points of the second - big polygons get more - standing on it, each turned at random.
Cameras, lights and render	**Camera from view**	Makes a free camera that looks exactly as the active viewport does, with the same field of view.
Cameras, lights and render	**Three-point lights**	Places key, fill and rim lights around the selection (or the origin) - a starting light for a look-dev render.
Cameras, lights and render	**Render size 1920 x 1080**	Sets the render output to 1920 x 1080 (Full HD).
Cameras, lights and render	**Lights on/off**	Switches every light in the scene off, or all back on.

Four scripts worth having on a dial

The same four are in the library; the code is printed here so you can see what runs and adapt it.

1. Pivot to bottom

Moves each selected object's pivot to the centre of its bottom face - so it sits on the grid and scales from the floor. Objects sit on the grid and scale from the floor - furniture, props, anything placed on a surface.

```
(
  local picked = selection as array
  if picked.count == 0 then
    format "MarkingForge: nothing selected\n"
  else
    for o in picked where (isValidNode o) do o.pivot = [o.center.x, o.center.y, o.min.z]
)
```

2. Select n-gons

Selects every polygon with five or more sides on the selected Editable Poly - the quickest topology check before subdividing. Before TurboSmooth or export: every polygon with five or more sides lights up, and the Listener says how many.

```
(
  local o = selection[1]
  if selection.count != 1 or (classOf o.baseObject) != Editable_Poly then
    format "MarkingForge: select ONE Editable Poly object\n"
  else
  (
    local found = for f = 1 to (polyop.getNumFaces o) where ((polyop.getFaceDeg o f) > 4) collect
f
    max modify mode
    modPanel.setCurrentObject o.baseObject
    subobjectLevel = 4
    polyop.setFaceSelection o (found as bitArray)
    redrawViews()
    format "MarkingForge: % polygon(s) with 5+ sides\n" found.count
  )
)
```

3. Reset XForm + collapse

Bakes rotation and scale into the mesh and clears the stack - the usual clean-up before exporting or before modifiers that misbehave on a scaled object. A scaled or rotated object misbehaves under Shell, Symmetry or Bevel, and exports with a transform the next program reads differently. This bakes it into the mesh.

```
(
  local picked = selection as array
  if picked.count == 0 then
    format "MarkingForge: nothing selected\n"
  else
    for o in picked where (isValidNode o) and (superClassOf o == GeometryClass) do (ResetXForm o;
maxOps.CollapseNode o off)
)
```

4. Smoothing on/off

Switches every TurboSmooth and MeshSmooth on the selected objects on or off - a fast low-poly / high-poly check. Flip between the cage and the smoothed result on every selected object at once - the low-poly / high-poly check you do a hundred times a day.

```
(
  local picked = selection as array
  local done = #()
  if picked.count == 0 then
    format "MarkingForge: nothing selected\n"
  else
    for o in picked where (isValidNode o) do (for m in o.modifiers where (classOf m ==
TurboSmooth or classOf m == MeshSmooth) and (findItem done m) == 0 do (append done m; m.enabled =
not m.enabled))
)
```

From the library: pick the dial, select a direction, **Script library...**, choose the script, **Put on direction**, **Save**. **Your own code:** select a direction, **Custom script...**, paste, OK, **Change label...** for a caption, **Save**.

A script with two versions (the rim)

A command with a caddy runs at once on an ordinary release and opens the caddy past the rim (or the other way round, chapter 2). A script can do the same: **Settings...** on a script item takes a second version *with parameters* - here the Extrude caddy of an Edit Poly modifier. Which version an ordinary release runs is chosen in the same window; releasing past the rim runs the other one.

Settings choice	Ordinary release	Past the rim
As the global setting (default: without parameters)	the plain code	the version with parameters
Always with parameters	the version with parameters	the plain code
Always without parameters	the plain code	the version with parameters

The library's **Extrude (Edit Poly)** is set up as *Always without parameters*: plain Extrude inside, the caddy past the rim.

```
// the plain code
(
  local m = modPanel.getCurrentObject()
  if classOf m != Edit_Poly then
    format "MarkingForge: no Edit Poly in the Modify panel\n"
  else
    m.EnterCommandMode #ExtrudeFace
)

// the version with parameters
(
  local m = modPanel.getCurrentObject()
  if classOf m != Edit_Poly then
    format "MarkingForge: no Edit Poly in the Modify panel\n"
  else
    m.PopupDialog #ExtrudeFace
)
```

Writing your own - the shape that works

Every library script has the same three habits; copy them and a script behaves like a built-in item:

- **Snapshot the selection first** - `selection as array`. A command that creates or deletes objects would otherwise change the very set being walked.

- **Say why nothing happened.** With nothing selected, print a line to the MAXScript Listener instead of failing silently.
- **Skip what cannot take the operation** - `where (isValidNode o)`, plus a class test where it matters - instead of stopping on the first light or camera.

```
(
  local picked = selection as array
  if picked.count == 0 then
    format "MarkingForge: nothing selected\n"
  else
    for o in picked where (isValidNode o) do ( /* your code - o is each selected object */ )
)
```

Scripts run with full rights, like an .mcr file. Paste only code you understand, and never code from a scene or a preset you have not read (chapter 13).

Where the gesture began: MF_ORIGIN_*

Before anything picked from a dial runs, MarkingForge sets three MAXScript globals to the point where the gesture began - the dial's centre, or where a flick started. A script or macroscript can create or place something there instead of at the scene's origin.

Global	Value
`MF_ORIGIN_SCREEN`	Point2 - desktop pixels.
`MF_ORIGIN_VIEW`	Point2 - pixels in the active viewport.
`MF_ORIGIN_WORLD`	Point3 - where the viewport ray through that point meets the world's Z=0 plane; the ray's start when it runs almost along the plane, `[0,0,0]` when it cannot be worked out.

They are set for a held or pinned dial, a flick or a tap, a list row, every item of a queue (the same point for all of them, taken at the release), and since 1.5.14 for the hotbox and Type-to-search too. Every pick replaces them; until the first pick of a session they are `undefined`, so test for that:

```
( local p = if MF_ORIGIN_WORLD != undefined then MF_ORIGIN_WORLD else [0,0,0]
  select (teapot pos:p) )
```

9. Shortcuts and mouse buttons

Keyboard

The standard scheme is Alt+1..8 (dials 1-8), Ctrl+Alt+1..8 (9-16), Shift+Alt+1..8 (17-24). The first start of MarkingForge in a 3ds Max gives every dial its standard key when no dial has one yet - however it was installed (1.1.0) - skipping keys something else uses and never touching your own shortcuts. It happens once per 3ds Max version and user; three small files in `plugcfg\MarkingForge` steer it:

File	Written by	Effect on the next start
`setup_shortcuts.flag`	The installer with *Set up the standard shortcuts on the next start* ticked; `Deploy-MarkingForge.ps1 -SetupShortcuts`	Free dials get their standard keys - even when this is not the first start. The file is then removed.
`skip_shortcuts.flag`	The installer with that box cleared; `Deploy-MarkingForge.ps1 -NoShortcuts`	Nothing is assigned. The file is then removed.
`shortcuts_first_start.done`	The first start itself, before it does the work	Marks the first start as done, so a failure is not repeated at every start.

Three ways to assign keys later:

- **MarkingForge > Set up standard shortcuts...** - free dials only, skips keys already in use.
- The editor: *This slot's shortcut* + **Apply shortcuts**, or **Fill the free slots...**
- 3ds Max's **Customize > Hotkey Editor** (also **MarkingForge > Assign keyboard shortcuts...**): the actions *MarkingForge: Menu 1..24*.

Switching a dial's set with a key. The category **MarkingForge** in the Hotkey Editor also holds *Menu 1 - next set* ... *Menu 24 - next set*: each switches its dial to the next set, the same switch as the *Next set* > row. A dial without sets says so in the status bar.

MarkingForge keeps its shortcuts in its own hotkey set, `MarkingForge_Hotkeys`, which contains all of your other shortcuts too. Every change is preceded by a backup in `plugcfg\MarkingForge\hotkeys\backup_*.hsx`.

Mouse buttons

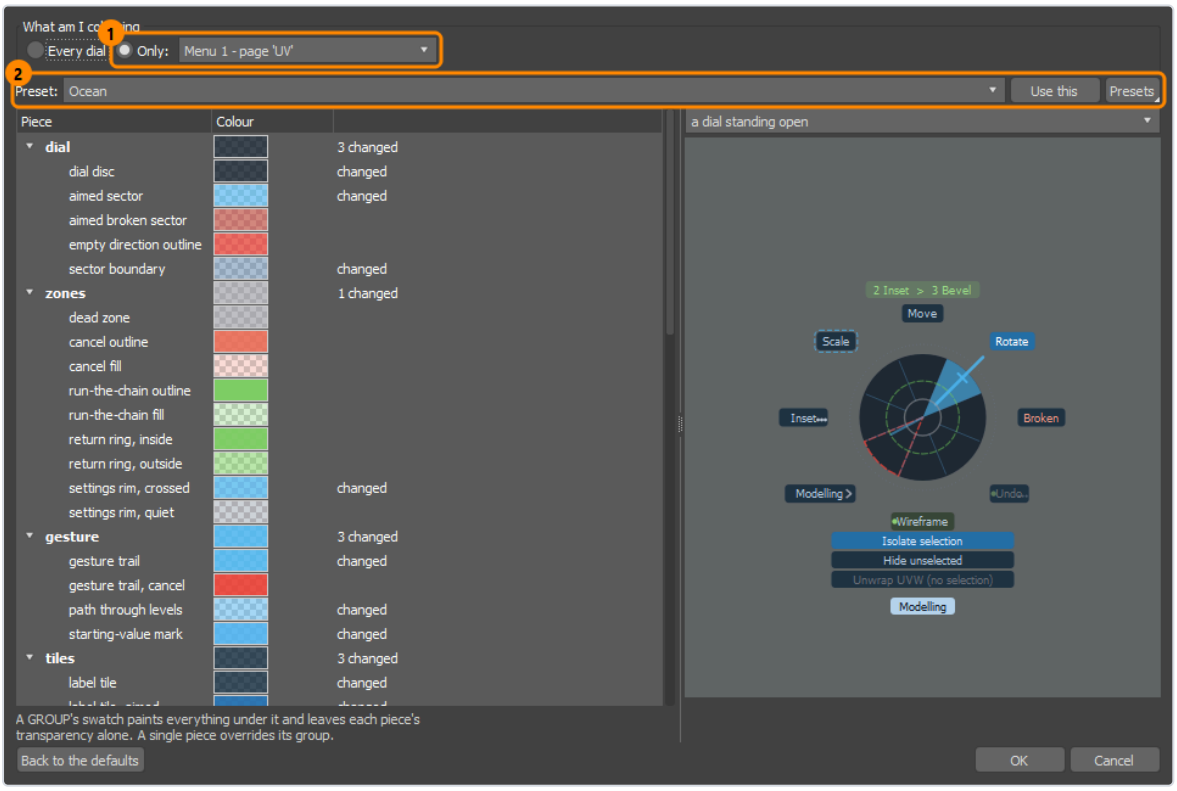
Button	Name in menus.json
(none - keyboard only)	
left button	`LMB`
middle button (wheel)	`MMB`
right button	`RMB`
thumb button 1 (back)	`XB1`
thumb button 2 (forward)	`XB2`

Modifiers: Ctrl, Alt, Shift - at least one is required, for example `"mouse": "Ctrl+Shift+MMB"`. A bare button would be taken from 3ds Max (left selects, middle pans, right opens the quad menu), so both the editor and the plugin refuse it. 3ds Max also uses some modifier+button combinations for navigation - if one collides with a habit of yours, pick another. With no dial bound to a button and no dial with sets (whose pages the wheel turns) the plugin installs no mouse hook at all.

10. Colours

Colours... in the editor. Two levels: a **group** colours everything in it (keeping each piece's transparency), a **single piece** overrides its group. **Colour...** under the directions colours one item's caption and tile. Only deviations from the defaults are saved.

Control	What it does
What am I colouring	**Every dial** - the palette the whole plugin draws with - or **Only** this dial, on top of it: a different colour per dial is the quickest way to know which one opened. For a dial with sets (1.4.0) **Only:** has a list: *Menu N - every page* or each page on its own (*Menu N - page 'UV'*, the one on the key marked); it starts on the page whose tab is open. A slot that shows the default menu cannot have colours of its own - give it its own menu first.
Preset: ... **Use this**	A ready-made look or one of yours (marked *(yours)*) paints what is chosen above - it replaces that palette as a whole look (1.4.0).
Presets	**Save these colours as a preset...**, **Delete this preset** (your own only), **Import a preset from a file...**, **Export this preset to a file...** (`.mfcolors`). An imported preset with a name you already have keeps both - *Mine (2)*.
Piece / Colour	The groups and their pieces, each with a swatch; a group says how many of its pieces are changed.
The moment (over the preview)	*a dial standing open*, *aiming at a direction*, *releasing would cancel*, *a chain is waiting* - some colours exist only in one of them.
Back to the defaults	Drops every colour in the current scope; the other scope is untouched.
OK / Cancel	OK keeps the colours in the editor until Save - only the scopes you edited are written (1.4.3); Cancel drops them.



Colours...: one page of Menu 1 chosen (1), painted with the Ocean preset (2).

The 30 ready-made dial looks (1.4.0 - 1.4.2): MarkingForge default, Graphite, Slate, Ocean, Violet, Rose, Ember, Mono, Sand - light tiles, High contrast, Indigo, Twilight, Steel, Crimson, Copper, Gold, Ice - light

tiles, Lavender - light tiles, Sapphire, Amethyst, Magenta, Coral, Sunset, Amber, Ruby, Storm, Carbon, Neon, Snow - light tiles, Peach - light tiles. The looks marked *light tiles* have dark captions on light tiles.

```
"theme": { "groups": { "dial": "#101820", "tiles": "#1c3a20" },
           "colors": { "tileFillSelected": "#3f8a4aff" } }
```

Colours are `#RRGGBB` or `#RRGGBBAA` (AA = opacity). The same `theme` can sit at the top of menus.json (all dials) or inside one menu.

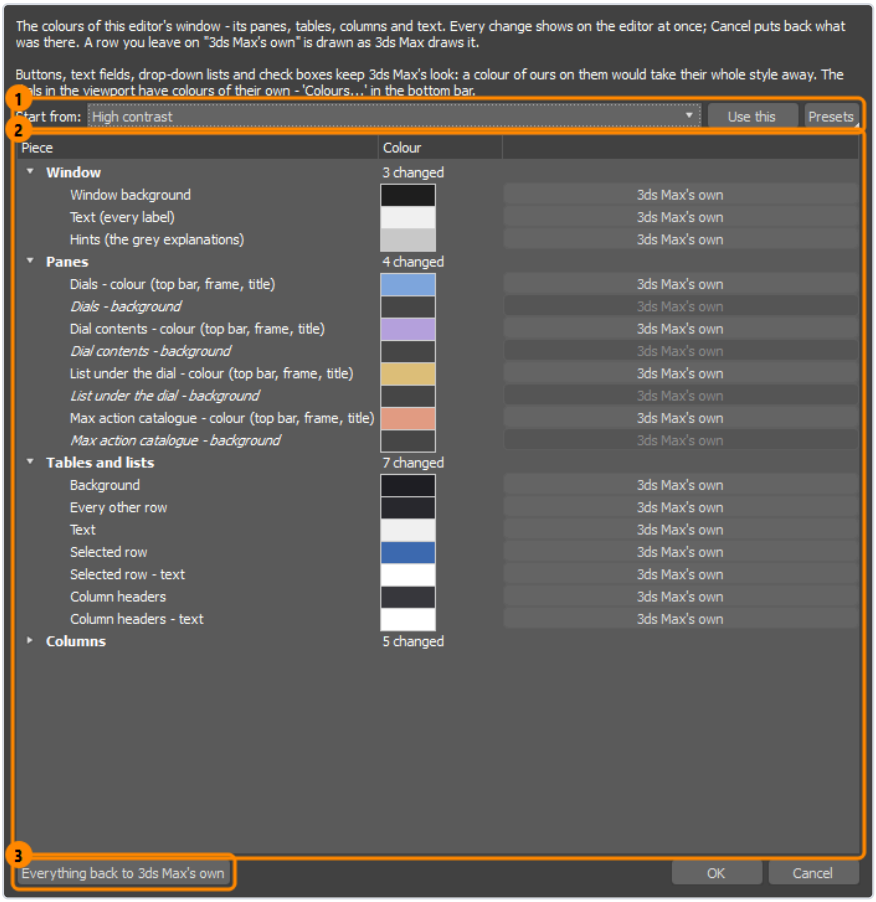
Group	Name	Piece	Default
dial	`dialFill`	dial disc	`#18181acd`
dial	`sectorFill`	aimed sector	`#508cc8a5`
dial	`sectorBroken`	aimed broken sector	`#be463ca5`
dial	`sectorEmpty`	empty direction outline	`#eb463cc8`
dial	`spoke`	sector boundary	`#96969b78`
zones	`deadZone`	dead zone	`#96969b96`
zones	`cancelRing`	cancel outline	`#eb6e5aeb`
zones	`cancelFill`	cancel fill	`#eb6e5a37`
zones	`queueRing`	run-the-chain outline	`#78cd5ff0`
zones	`queueFill`	run-the-chain fill	`#78cd5f46`
zones	`returnRingNear`	return ring, inside	`#78cd5feb`
zones	`returnRingFar`	return ring, outside	`#78cd5f82`
zones	`settingsRingActive`	settings rim, crossed	`#96c8fabe`
zones	`settingsRingIdle`	settings rim, quiet	`#7882915a`
gesture	`trail`	gesture trail	`#ffbe5ae1`
gesture	`trailCancel`	gesture trail, cancel	`#eb463cf0`
gesture	`trailBack`	path through levels	`#ffbe5a82`
gesture	`valueMark`	starting-value mark	`#ffd278e6`
tiles	`tileFill`	label tile	`#18181acd`
tiles	`tileFillSelected`	label tile, aimed	`#3c6ea5eb`
tiles	`tileFillBroken`	label tile, broken	`#963730eb`
tiles	`tileFillChecked`	label tile, toggle on	`#34402ed7`
tiles	`tileDefaultOutline`	chosen-for-you outline	`#96c8fad2`
tiles	`tileQueuedDot`	already-queued dot	`#78cd5fe6`
tiles	`tileCheckDot`	toggle dot	`#96d778f0`
tiles	`tileCheckDotDim`	toggle dot, unavailable	`#6e8c64c8`
tiles	`tileArrow`	submenu arrow	`#c8c8d2be`
tiles	`tileArrowSelected`	submenu arrow, aimed	`#ffffffeb`
tiles	`tileEllipsis`	settings dots	`#d7e1f0d7`

tiles	`tileEllipsisSelected`	settings dots, aimed	`#ffffffff5`
tiles	`tileEllipsisIdle`	settings dots, dimmed	`#969eac6e`
tiles	`tileEllipsisIdleSel`	dimmed dots, aimed	`#ffffff69`
text	`textLabel`	label	`#d7d7ded7`
text	`textLabelSelected`	label, aimed	`#ffffffff`
text	`textBroken`	label, broken	`#ffa08cf5`
text	`textBrokenSelected`	label, broken and aimed	`#ffebe6ff`
text	`textDim`	label, unavailable	`#828288be`
text	`textDimSelected`	unavailable and aimed	`#aaaaafeb`
list	`listFill`	list row	`#18181acd`
list	`listFillPointed`	list row, pointed at	`#4682c8e1`
list	`listText`	list row text	`#e8ecf2d7`
list	`listTextPointed`	list text, pointed at	`#e8ecf2ff`
list	`listTextDim`	list text, unavailable	`#96969ba0`
footer	`footerTile`	footer tile	`#e8ecf2e1`
footer	`footerText`	footer text	`#000000eb`
chain	`chainTile`	chain tile	`#78cd5f37`
chain	`chainText`	chain text	`#a0e68cf0`
hotbox	`hotboxCentre`	hotbox centre dot	`#c8c8d278`
hotbox	`hotboxTitleFill`	menu title tile	`#18181ad2`
hotbox	`hotboxTitleFillOpen`	menu title, open	`#3c6ea5eb`
hotbox	`hotboxTitleText`	menu title text	`#d7d7ded7`
hotbox	`hotboxTitleTextOpen`	menu title text, open	`#ffffffa`
hotbox	`hotboxPanel`	expanded list panel	`#121214e6`
hotbox	`hotboxRowFillHot`	hotbox row, pointed at	`#3c6ea5eb`
hotbox	`hotboxRowText`	hotbox row text	`#d2d2dad2`
hotbox	`hotboxRowTextHot`	hotbox row text, hot	`#ffffffa`

The editor's own colours

Editor colours... colours the editor window, not the dials: the window (ground, text, hints), each pane (colour and ground), the tables and lists (ground, every other row, text, selected row, column headers) and each column of each table on its own (text and ground) - four groups: *Window*, *Panes*, *Tables and lists*, *Columns* (1.3.0). Every change shows on the editor at once; Cancel puts back what was there. **Start from:** offers *3ds Max's own (nothing changed)* and 21 ready-made looks - High contrast, Calm grey, Graphite, Midnight blue, Warm sepia, Plum, Ocean, Charcoal, Steel blue, Indigo night, Mocha, Rose dust, Nord, Sapphire, Amethyst, Carbon, Storm, Burgundy, Slate grey, Dusk, Espresso - plus presets of your own, with the same **Use this** and **Presets** as the dials' colours (1.4.0). **3ds Max's own** on a row drops that one colour; **Everything back to 3ds Max's own** drops them all. OK saves at once - the editor's colours are not

part of your dials and need no Save. Only what you chose is saved, in `editor_colors.json` beside menus.json:



Editor colours...: starting points (1), every piece of the window (2), back to 3ds Max's own (3).

```
{ "kind": "MarkingForge editor colours", "version": 1,
  "colors": { "views.bg": "#1f2023", "col.dir.0.text": "#9cc4ff" } }
```

Editor colours are `#RRGGBB` (no opacity); a key the editor does not know, or a value it cannot read, loses that one colour, not the rest. Buttons, text fields, drop-down lists and check boxes keep 3ds Max's look.

Presets and pages

Both colour editors have **presets**: ready-made ones and your own. Your own are kept in `dial_color_presets.json` and `editor_color_presets.json` beside menus.json; one preset travels as a `.mfcolors` file, which says whether it is for the dials or the editor and is refused by the other. A presets file that cannot be read is left as it is and said so (1.4.3). The dials' ready-made looks keep each piece's transparency and change only its colour.

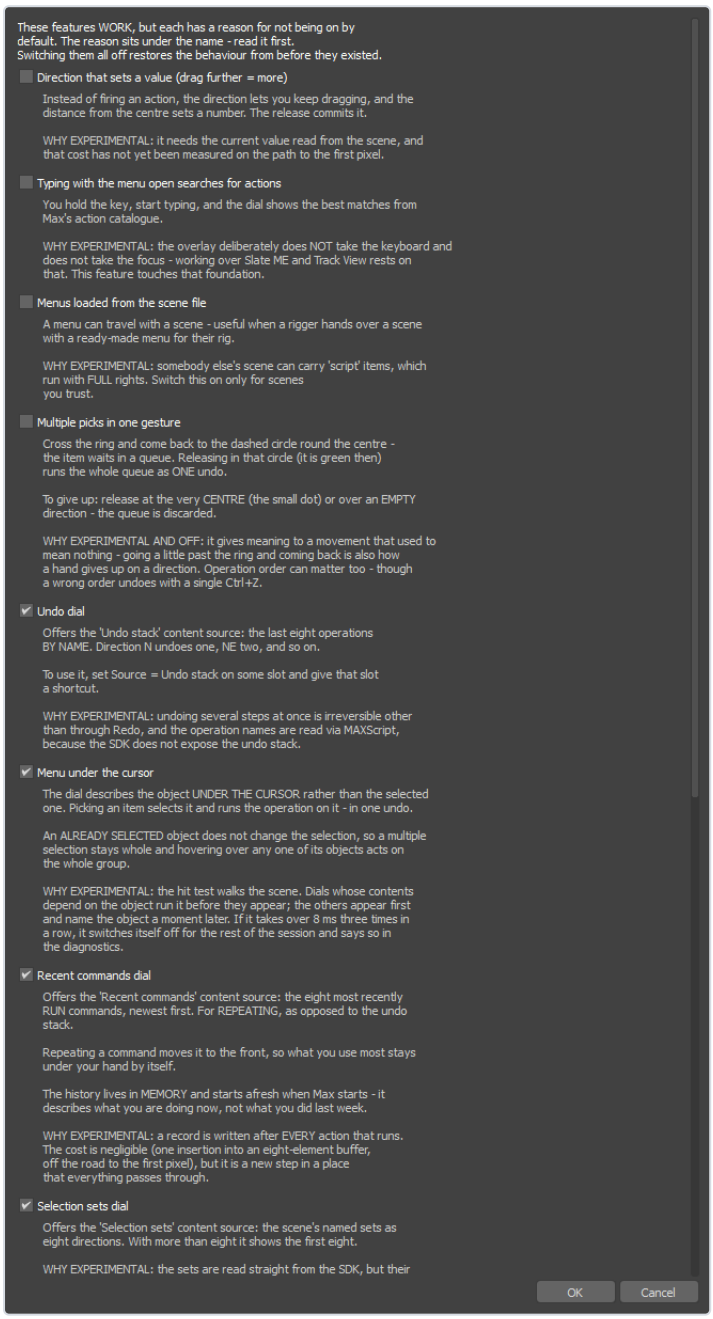
```
{ "kind": "MarkingForge colour preset", "for": "dial", "version": 1, "name": "Mine",
  "colors": { "tileFillSelected": "#3f8a4a" } }
```

A page's palette is the `theme` inside that set in `dialSets`, and it travels with the set when the set goes on the key. A palette cascades: every dial > the dial or page > its context variants > their submenus - each level changes only what it names (1.4.0; before, a dial's own colours stopped at its base menu).

11. Experimental features

Features that work but each have a reason not to be forced on everyone. Switch them in the editor (**Experimental features...**) or in **MarkingForge > Experimental features**; the tick in the menu shows the state. Switching all of them off restores the behaviour from before they existed.

A tick in the MarkingForge menu saves menus.json at once. An editor that is open at that moment then refuses to Save until you reload (see Save in chapter 5).



The experimental features dialog.

> **Direction that sets a value (drag further = more)** `dragValue` OFF BY DEFAULT
>
> Instead of firing an action, the direction lets you keep dragging, and the distance from the centre sets a number. The release commits it.
>
> WHY EXPERIMENTAL: it needs the current value read from the scene, and that cost has not yet been measured on the path to the first pixel.

Typing with the menu open searches for actions `gestureSearch` OFF BY DEFAULT

You hold the key, start typing, and the dial shows the best matches from Max's action catalogue.

WHY EXPERIMENTAL: the overlay deliberately does NOT take the keyboard and does not take the focus - working over Slate ME and Track View rests on that. This feature touches that foundation.

Menus loaded from the scene file `sceneMenus` OFF BY DEFAULT

A menu can travel with a scene - useful when a rigger hands over a scene with a ready-made menu for their rig.

WHY EXPERIMENTAL: somebody else's scene can carry 'script' items, which run with FULL rights. Switch this on only for scenes you trust.

Multiple picks in one gesture `multiPick` OFF BY DEFAULT

Cross the ring and come back to the dashed circle round the centre - the item waits in a queue. Releasing in that circle (it is green then) runs the whole queue, in order.

Chain commands that finish at once (Ring, Flip, Connect...). Inset, Extrude, Bevel or Chamfer wait for you: the queue stops there - drag, then RIGHT-CLICK, and the next queued command starts by itself. Commands that finish at once undo together with one Ctrl+Z; each one that waited for you is its own Ctrl+Z.

To give up: release at the very CENTRE (the small dot) or over an EMPTY direction - the queue is discarded.

WHY EXPERIMENTAL AND OFF: it gives meaning to a movement that used to mean nothing - going a little past the ring and coming back is also how a hand gives up on a direction. Operation order can matter too - though a wrong order undoes with a single Ctrl+Z.

Undo dial `undoDial` ON BY DEFAULT

Offers the 'Undo stack' content source: the last eight operations BY NAME. Direction N undoes one, NE two, and so on.

To use it, set Source = Undo stack on some slot and give that slot a shortcut.

WHY EXPERIMENTAL: undoing several steps at once is irreversible other than through Redo, and the operation names are read via MAXScript, because the SDK does not expose the undo stack.

Menu under the cursor `hoverTarget` ON BY DEFAULT

The dial describes the object UNDER THE CURSOR rather than the selected one. Picking an item selects it and runs the operation on it - in one undo.

An ALREADY SELECTED object does not change the selection, so a multiple selection stays whole and hovering over any one of its objects acts on the whole group.

WHY EXPERIMENTAL: the hit test walks the scene. Dials whose contents depend on the object run it before they appear; the others appear first and name the object a moment later. If it takes over 8 ms three times in a row, it switches itself off for the rest of the session and says so in the diagnostics.

Recent commands dial `recentDial` **ON BY DEFAULT**

Offers the 'Recent commands' content source: the eight most recently RUN commands, newest first. For REPEATING, as opposed to the undo stack.

Repeating a command moves it to the front, so what you use most stays under your hand by itself.

The history lives in MEMORY and starts afresh when Max starts - it describes what you are doing now, not what you did last week.

WHY EXPERIMENTAL: a record is written after EVERY action that runs. The cost is negligible (one insertion into an eight-element buffer, off the road to the first pixel), but it is a new step in a place that everything passes through.

Selection sets dial `selectionSetsDial` **ON BY DEFAULT**

Offers the 'Selection sets' content source: the scene's named sets as eight directions. With more than eight it shows the first eight.

WHY EXPERIMENTAL: the sets are read straight from the SDK, but their NAMES are written by the user and end up in the MAXScript code that performs the selection. Quoting is handled (a quote or backslash in a name does not break the code), but the feature is waiting to be confirmed in real work.

Modifier under the cursor `modifierUnderCursor` **ON BY DEFAULT**

A dial with the 'Modifier stack' source shows the stack of the object UNDER THE CURSOR rather than the selected one. Requires 'Menu under the cursor' - without it there is nowhere to get the object from.

WHY EXPERIMENTAL: the same risk as the menu under the cursor - the dial talks about an object the user did not select, so that must be visible. The object's name is in the dial header, marked [under cursor].

Operation-pair learning `pairLearning` **ON BY DEFAULT**

The plugin remembers what follows what and sets the dial's DEFAULT direction to the most frequent successor of the last operation - the one a TAP of the key will run (and Enter, when the dial is pinned). A pair counts from its second repetition; a 'tap' item of the dial always wins over the suggestion.

WHAT IT DOES NOT DO: it does not move items. A menu that reorders itself turns a remembered gesture into reading, and muscle memory is the whole point of a marking menu.

WHY EXPERIMENTAL: the memory is SESSION-ONLY and nobody has yet measured whether the hints land. The hit counter is in 'Configuration status'.

Chain as an item `chainAsItem` **ON BY DEFAULT**

After you run a chain (multiple picks) the plugin REMEMBERS it. You then store it with 'Save last chain...' in the MarkingForge menu.

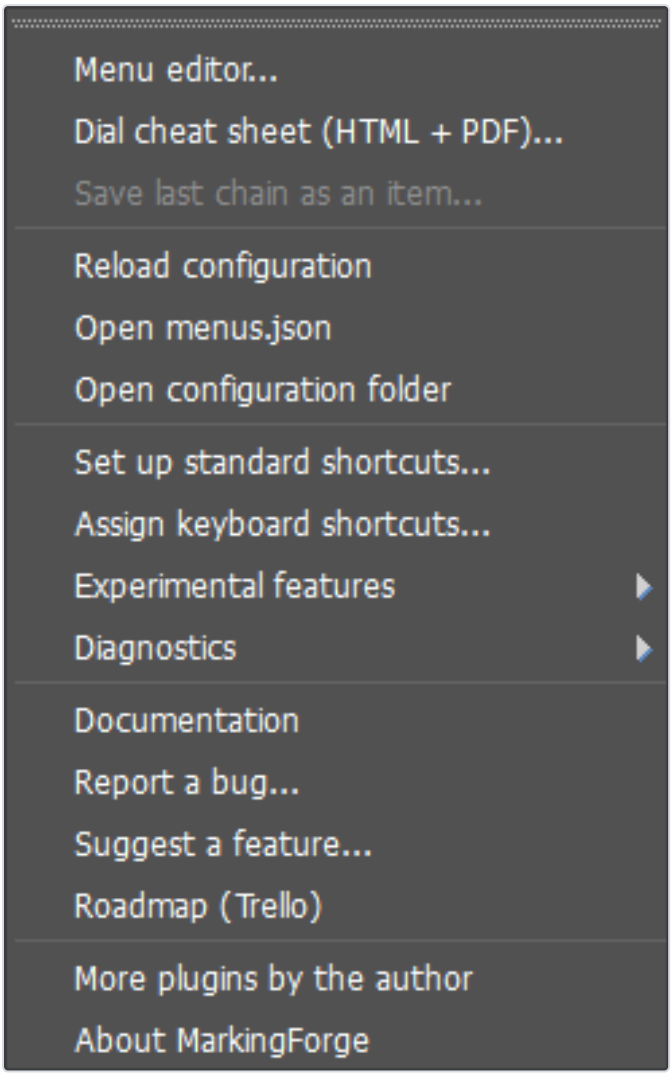
DELIBERATELY NO PROMPT AFTER EVERY GESTURE: a dialog popping up after a chain would turn a feature that speeds work up into one that interrupts it.

WHY EXPERIMENTAL: the result is stored as a 'script' item, and replay works only for actions and macros - a chain containing a 'value' item or a submenu will not save at all, and says so.

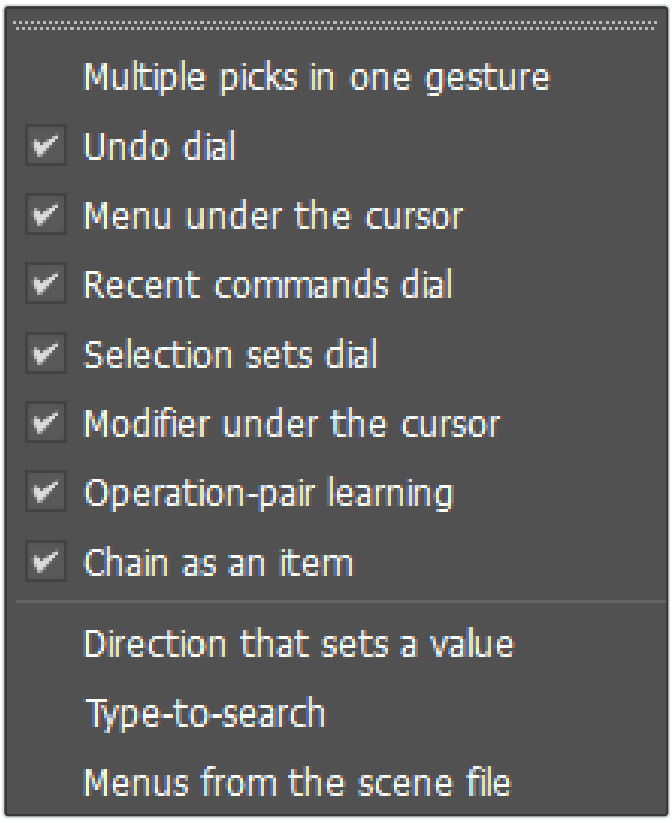
An ordinary release shows the action's settings `showItemSettings` OFF BY DEFAULT

Ticked, picking Inset, Extrude, Bevel... runs the command and opens its caddy, as the quad menu's settings icon does. Cleared - the default since 1.5.17 - it runs at once, as its Modify-panel button does, and releasing past the rim opens the caddy. This is only the default: crossing the rim reverses it, and **Settings...** can change it per item.

12. The MarkingForge menu



The menu.



Experimental features.

Every command is also an action in the category **MarkingForge**, so it can go on a toolbar, into a menu or quad menu (**Customize > Menu Editor**), or get a hotkey.

Command	What it does
Menu editor...	Opens a window where you assign actions to dial directions and set keyboard shortcuts.
Dial cheat sheet (HTML + PDF)...	Collects every dial into one document to read and to print. Along the way it checks which dial would open empty.
Save last chain as an item...	Takes the last chain you ran (multiple picks) and stores it as ONE direction in a dial of your choice.
Reload configuration	Reads menus.json again. Useful when the file was edited outside Max.
Open menus.json	Opens the configuration file in your default text editor. The plugin picks up changes by itself.
Open configuration folder	Opens the folder holding menus.json, the shortcut backups and the settings in Explorer.
Assign keyboard shortcuts...	Opens 3ds Max's Hotkey Editor. The dials are the actions 'MarkingForge: Menu 1' ... 'Menu 24' - type MarkingForge into its search field.
Set up standard shortcuts...	Gives every dial without a shortcut its standard one: Alt+1..8, Ctrl+Alt+1..8, Shift+Alt+1..8. Keys already in use are skipped; your own shortcuts are not touched.

Configuration status	Shows which file is loaded, how many menus it holds, whether it has errors, and the state of the experimental features (including how often pair-learning suggestions were taken).
About MarkingForge	Shows the plugin's version, its configuration file, and links to support and to more plugins by the author.
Documentation	Opens the folder with the MarkingForge guides (QuickStart, Reference, Installation and more).
Report a bug...	Copies your 3ds Max and MarkingForge details to the clipboard and opens a window with links to the bug report form and the e-mail address.
Suggest a feature...	Send an idea or a feature request to forgeplugins@gmail.com.
Roadmap (Trello)	Opens the MarkingForge features roadmap on Trello in your web browser - what is planned and in progress.
More plugins by the author	Opens https://forgeplugins.tech in your web browser.
Multiple picks in one gesture	Cross the ring and come back to the dashed circle - the item waits in a queue. Releasing in the circle runs the queue in order; Inset, Bevel... wait for a right-click.
Undo dial	Offers the "undo" menu source: the last eight operations by name. Assign it to a slot in the menu editor.
Menu under the cursor	The dial describes the object UNDER THE CURSOR rather than the selected one. Picking an item selects that object and runs the operation on it - in a single undo.
Direction that sets a value	A "value" direction turns into a slider - the gesture radius sets the number.
Type-to-search	Typing while the dial is open turns it into a search over Max's actions.
Menus from the scene file	CAUTION: menus carried in by somebody else's scene. Script items are rejected, but switch this on deliberately all the same.
Modifier under the cursor	A dial with the 'Modifier stack' source shows the stack of the object UNDER THE CURSOR rather than the selected one. Requires 'Menu under the cursor' - without it there is nowhere to get the object from.
Operation-pair learning	The plugin remembers what follows what and sets the dial's DEFAULT direction to the most frequent successor. It does NOT move items around - muscle memory rests on that. Session memory only.
Chain as an item	After you run a chain (multiple picks) the plugin remembers it. You then store it with 'Save last chain...' - with no dialog popping up after every gesture.
Recent commands dial	Offers the 'Recent commands' content source: the last eight you ran, newest first. For REPEATING, as opposed to the undo stack.
Selection sets dial	Offers the 'Selection sets' content source: the scene's named sets as eight directions.
Menu 1 - next set ... Menu 24 - next set	Switches that dial to its next set - one action per dial, for a key of its own. Sets are made in the menu editor.

Diagnostics are in **MarkingForge > Diagnostics**: **Event log**, **Clear the event log** and **Configuration status** (chapter 15); they print to the MAXScript Listener and change no dial.

13. Menus carried by scenes

A scene file can carry its own dials - for example a rigger handing over a character with a ready menu for its controls. This needs the experimental **sceneMenus** (off by default).

The mode is chosen in the editor's **Experimental features...**, in the box *When an opened scene carries its own menus*, which also says what the open scene carries and has two buttons: **Use this scene's menus** and **Back to my configuration** (your file stays untouched either way).

Mode (sceneMenuMode)	Behaviour
Ask - load them, but do NOT use them without my consent (recommended)	The scene's menus are loaded and counted, but the dials STILL show your configuration. The Listener reports that the scene carries something, and you switch with the button below or the sceneMenuUse command.
Always use the scene's menus when it carries any	For a studio where project scenes ARE meant to bring their own menus. CAUTION: this means opening somebody else's file REPLACES your dials without asking. Script items are rejected regardless.
Never - do not even look inside scenes	The plugin does not read menu data from scene files at all. The most cautious setting.

Command	What it does
`MarkingForge.sceneMenuSave <slot>`	Stores a copy of that slot's menu in the open scene. Save the scene to keep it.
`MarkingForge.sceneMenuInfo()`	Says whether the scene carries menus and whether they are in use.
`MarkingForge.sceneMenuUse true|false`	Switches between the scene's menus and yours.
`MarkingForge.sceneMenuClear()`	Removes the menus from the scene.

> Script items in a scene's menus are always rejected - somebody else's scene must not run code with your rights.

14. The configuration file (menus.json)

MarkingForge > Open menus.json. Everything the editor does is stored here, and the file can be edited by hand: the plugin reloads it when it is saved. A mistake disables only the item it is in and is reported by **Diagnostics > Configuration status** - the rest keeps working.

Top level

Key	Meaning
`schema`	File format version - `1`. Leave as written; another number refuses the whole file.
`menus`	An object with keys `"1"` ... `"24"` - each a menu.
`defaultMenu`	The menu shown by slots without their own.
`experimental`	The experimental switches that differ from the defaults, e.g. `{ "gestureSearch": true }`.
`theme`	Colours for all dials (chapter 10).
`showItemSettings`	`true` = an ordinary release opens the caddy; missing = `false`, the command runs at once (since 1.5.17; until 1.5.16 missing meant `true`).
`rimDistance`	The settings ring's distance past the farthest caption in pixels, `0` to `200`; missing = `20` (1.5.10). A value outside is clamped and reported by Configuration status.
`sceneMenuMode`	`ask` (default), `always` or `never`.
`dialSets`	The dials' other sets (1.0.9): for each slot with sets, the name of the set on the key (`active`), their `order` and the other sets' menus (`sets`). What the key opens stays in `menus`; since 1.2.0 the plugin also reads `dialSets` for the pages the mouse wheel turns - the sets after the active one in `order`, wrapping round. A malformed entry costs the pages, never the dial. Putting a set on the key moves it into `menus` and the one there into `sets`, in one write.

```
"dialSets": { "1": { "active": "UV",
                     "order":  ["Modelling", "UV", "Retopo"],
                     "sets":   { "Modelling": {menu}, "Retopo": {menu} } } }
```

A menu

Key	Meaning
`name`	The dial's name.
`items`	Directions `N NE E SE S SW W NW`, each an item (chapter 8). A missing direction is empty.
`list`	An array of up to twelve items drawn under the dial.
`rules`	An array of variants: `{ "when": {...}, "menu": {...} }`. First match wins.
`source`	`config` (the default), `modifiers`, `undo`, `recent` or `selectionSets`.
`layout`	`"wheel"` (the default, a dial) or `"hotbox"`.
`tap`	A direction name (`"N"`) or a whole item run on a tap.
`mouse`	A mouse binding, e.g. `"Alt+MMB"`. Belongs to the slot: it stays when another dial or set is put on it, and the default menu cannot carry one.
`theme`	Colours for this dial (or, inside `dialSets`, this page) only.

Conditions (`when`)

Key	Example
`class`	`"Editable Poly"`
`superClass`	`"shape"` , `"light"` , `"camera"` , `"geometry"` , `"helper"`
`modifier`	`"Edit_Poly"` (open in the Modify panel)
`hasModifier`	`"TurboSmooth"` (anywhere in the stack)
`subobject`	`1` … `5` , `0` = object level
`window`	`"slate"` , `"trackView"` … (chapter 6)
`selection`	`"none"` , `"one"` , `"many"` , `"any"` or a number
`name`	`"Rig_*"`
`layer`	`"Characters"`

Item fields common to all types

Key	Meaning
`type`	`action` , `macro` , `script` , `submenu` , `value`
`label`	Caption on the dial (optional for action and macro).
`settings`	`true` / `false` - open the caddy on an ordinary release.
`codeSettings`	Script only: the version *with parameters*, run instead of `code` whenever the item wants its parameters (see `settings` ; reversed past the rim).
`color` , `tile`	This item's caption and tile colour.

A complete small example

```
{
  "menus": {
    "3": {
      "name": "My tools",
      "items": {
        "N": { "type": "action", "table": 0, "id": "40021", "label": "Select all" },
        "S": { "type": "action", "table": 0, "id": "40043", "label": "Select none" },
        "E": { "type": "macro", "category": "Modifiers", "name": "TurboSmoothMod" },
        "W": { "type": "script", "label": "Box", "code": "startObjectCreation Box" }
      },
      "rules": [
        { "when": { "class": "Editable Poly", "subobject": 2 },
          "menu": { "name": "Edges", "items": {
            "N": { "type": "macro", "category": "Editable Polygon Object", "name":
"EPoly_Connect" } } } }
      ]
    }
  }
}
```

> Hover a command in the editor's catalogue to see its `table` and `id` - the fastest way to write actions by hand.

A dial file (.mfdial) and a folder of dials

A `.mfdial` (Save dial..., Save every dial..., the dial library) is plain JSON holding one menu exactly as in `menus` - without the slot's `mouse`:

```
{ "kind": "markingforge-dial", "version": 1, "name": "My tools",
  "menu": { "name": "My tools", "items": { ... } } }
```

A folder made by **Save every dial...** holds *NN name.mfdial* files and, since 1.4.4, `MarkingForge dials.json` - which file is which dial, which set each dial showed, the sets' order, the dials that showed the default menu, and the palette of every dial (the top-level `theme`); since 1.5.55 (version 2) also `config`, the whole menus.json as it was. **Load every dial...** reads it; without it the file names are used.

```
{ "kind": "markingforge-dial-folder", "version": 2, "theme": { ... }, "config": { ... },
  "dials": [ { "file": "01 Modelling.mfdial", "slot": 1, "name": "Modelling",
               "shown": true, "position": 0, "inherited": false }, ... ] }
```

15. MAXScript commands

Type these in the MAXScript Listener (F11). They read state or change settings; none of them changes the scene except where said.

Command	Returns / does
`MarkingForge.version()`	The plugin version. "Unknown property" = the plugin is not loaded.
`MarkingForge.configSummary()`	The loaded file, the number of menus, errors and warnings - and whether `MARKINGFORGE_CONFIG_DIR` was used, or why not. The same as **Diagnostics > Configuration status**.
`MarkingForge.configPath()`	The full path of menus.json.
`MarkingForge.reloadConfig()`	Reads menus.json again now.
`MarkingForge.menuModel <slot>`	What dial <slot> holds, resolved: each item and whether it exists in this 3ds Max.
`MarkingForge.contextInfo()`	The class, kind, modifier and sub-object level the plugin sees now - the words to put into a condition.
`MarkingForge.windowInfo()`	The window under the cursor and the word for a window condition.
`MarkingForge.windowCatalog()`	Every window open now, with its word.
`MarkingForge.eventLog()`	The last gestures: which dial and variant opened, what ran or why it cancelled, how long it took. Line kinds: **START_OVERRIDE** (a dial key went down), **END_OVERRIDE** (its release), **EXECUTE**, **FOCUS** (a window took the keyboard, 1.0.9) and **RELEASE_LOST** (a release 3ds Max did not report, 1.4.4). The same as **Diagnostics > Event log**.
`MarkingForge.clearEventLog()`	Empties the event log.
`MarkingForge.statsLine()`	Timing of the last dial opening, and whether the keyboard hook is installed (with the dial closed it must say no).
`MarkingForge.mouseInfo()`	Mouse bindings and whether the mouse hook is installed.
`MarkingForge.usageInfo()` / `usageReset()`	The usage counters in one sentence / clear them and `usage.json`.

`MarkingForge.experimentalInfo()`	Which experimental features are on.
`MarkingForge.experimentalGet "name"`	Whether one feature is on.
`MarkingForge.experimentalSet "name" true|false`	Switches a feature and saves the file; returns what happened, errors included.
`MarkingForge.hotboxInfo()`	The hotbox's snapshot of the main menu.
`MarkingForge.lastChainDesc()` / `lastChainCode()`	The last chain you ran, as text / as MAXScript.
`MarkingForge.snapshotMenu <slot> @"C:\\dial.png" -1`	Saves a picture of dial <slot> as it would open now (the last number 0-7 draws that direction aimed, -1 at rest), cropped, on a transparent ground. Runs nothing and counts no usage; returns `OK <w>x<h> <menu>` or `ERROR: ...`.
`MarkingForge.snapshotMenuEx <slot> @"C:\\dial.png" "aim=2;dist=200"`	The same picture with the dial in a chosen state; options as `key=value;key=value`: `aim=0..7`, `dist=` pixels from the centre (past the rim lights the dotted circle), `queue=A|B` (a waiting chain), `queued=0,2`, `default=0..7` (the outlined tile), `search=` text, `page=` 0-based page, `rim=` 0..200 (the settings ring's distance to draw with instead of yours, 1.5.10 - and the ring is then drawn even on a level without a command that has settings).
`MarkingForge.rimInfo()`	`rimDistance=N overlay=N`: the settings ring's distance in your configuration and the one the dial window is set to (1.5.10).
`MarkingForge.sceneMenu...`	Chapter 13.

For diagnosis and for the editor

The interface holds more functions, typed the same way (`MarkingForge.qtVersions()`). Some measure the plugin, some are how the menu editor talks to it; none changes the scene. They are listed so that nothing in `showInterface MarkingForge` is a mystery - you do not need them for everyday work.

Command	Returns / does
`qtVersions()`	The Qt version the plugin was built against and the one 3ds Max loaded.
`perfSeries()` / `perfReset()`	Every gesture that opened a dial since the reset (1.5.11; the last 4000), in blocks of 25: opening, first pixel, release, the cursor ticks while held (a tick includes the repaint) and the paints - medians and the worst tick - so a slowdown over a session shows as a

	trend; a hold of 10 s or more also per 5 s. Then the HITCHES with the time of day: a held tick over 25 ms (split into the guards, the cursor read and the repaint), an opening over 40 ms, a release over 60 ms. When a dial feels slow, run it and send the result with the bug report.
`actionsRegistered()`	Whether the MarkingForge actions (Menu 1..24) are registered.
`firstPaintMs()`	Time to the first pixel of the last dial, in ms (-1 while the dial window does not exist yet).
`paintSeries()` / `clearPaintSeries()`	Recent dial openings broken down by phase / empty that series (Diagnostics > Clear the event log also does).
`descendSeries()`	Times of submenu descents, from crossing the ring to the new level's first pixel.
`ruleBench <repeats>`	The cost of evaluating the context variants.
`hoverBench <repeats>`	The cost of finding the object under the cursor in the active viewport.
`searchInfo()`	The type-to-search index: how many actions, from how many tables, how long it took to build.
`hotboxRebuild()`	Rebuilds the hotbox's snapshot of the main menu; returns how long it took (ms).
`caddyDiag()`	What happened on the last attempt to open a settings dialog (caddy) - read from `caddy_diag.txt`.
`showTestOverlay <ms>` / `hideTestOverlay()`	Shows a test dial at the cursor for that long / hides it.
`isOverlayVisible()` / `overlayHasFocus()`	Whether the dial is on screen / whether it holds the keyboard focus (it must not).
`setRaiseMode <0-3>` / `getRaiseMode()`	How the dial window is raised - a measurement switch; leave it alone.
`setLabelMaxWidth <px>` / `getLabelMaxWidth()` / `overlaySide()`	The widest caption tile, and the dial window's size it leads to - for measurements.
`usageJson()`	The usage counters as JSON - the editor's Usage column.
`defaultMenusJson()`	The 24 built-in dials as JSON - what **Load default dials...** and **From a template...** offer; works even when menus.json is broken.
`themeCatalogJson()`	Every colour the dial draws with: name, group, description, default - the editor builds **Colours...** from it.
`actionCatalog()`	Every action of every action table in one string - the editor's catalogue.
`pauseGestures <ms>`	Switches the dials off for up to 10 seconds; the editor's **Set shortcut** window renews it while it is open.
`keyCapture <hwnd> <ms>` / `keyCaptureRead()` / `keyCaptureTimings()`	The **Set shortcut** window's key capture: start it for a window, read the keys pressed, its timings.

16. Tips and tricks

Learn by flicking, not by reading

Use the held dial for a few days, then start moving before it appears. Speed comes from never waiting for the drawing.

Put opposites opposite

Grow N / Shrink S, Hide S / Unhide N. The hand learns one movement and its reverse - the built-in dials follow this everywhere.

Keep NW as "back"

In every sub-object variant NW returns to object level. Reuse the convention in your own variants.

Eight is the limit, not the target

Put rarely used items in the list under the dial; directions are for what the hand should remember.

Short labels

Change label... with two words beats a cut-off 3ds Max name.

Colour the dial, not the item

A different dial colour per slot tells you which dial opened before you read a word. Colour single items only to warn - a red caption on a destructive command.

Specific variants first

Edit Poly at Vertex above Edit Poly above Editable Poly. When a variant never shows, a more general one above it is catching it first.

Let the context do the work

One key that changes with the sub-object level beats five keys. Look at the built-in Modelling dial before building your own.

Window variants

The same key can do one thing in the viewport and another over Slate or Track View - the built-in Materials, Animation and Scene dials show how.

Cross the rim for settings

Need the Chamfer caddy once? Move past the ring. Need it every time? Settings... on that item.

Chain several picks

Switch on *Multiple picks* first. Out past the ring and back to the dashed circle queues an item; release there runs the queue in order as one undo, the dot in the middle cancels. Chain commands that finish at once (Ring, Loop, Flip, Connect, Weld, a modifier). One that waits for a drag (Inset, Extrude, Bevel, Chamfer; on splines Fillet, Chamfer, Outline - also under the Edit Poly, Edit Spline and Edit Mesh modifiers) stops the queue: drag, right-click, and the next queued command starts (since 1.5.44; the status line names it from 1.5.45). While it waits: Select, Move, Rotate or Scale ends the tool like a right-click, Esc does not end it (3ds Max keeps it on), a pan or zoom keeps the queue, another tool, a change in the selected objects (another, one more, one fewer), a new gesture or a new scene drops the rest - the status line says so; each such step is its own undo. A value direction you let go over is the queue's last step: written after the tool, or dropped with the rest (1.5.50). The event log shows QUEUE lines for waiting, resuming and dropping. Save a chain you repeat as a single item with **Save last chain as an item...** (one that waited for a tool is not offered).

Let the dial suggest the next step

With pair learning on, MarkingForge notices what you run after what - Inset, then Extrude. Once a pair has repeated twice, the next command is the dial's outlined tile and a **tap** of the key runs it without aiming. Nothing on the dial moves; the memory lasts for the session, and **Diagnostics > Configuration status** shows how often the suggestions were taken.

Point instead of selecting

With *Menu under the cursor* the dial answers for the object you point at and the command runs on it - no click to select first. Pointing at one of several selected objects keeps the whole selection.

Type when you cannot find it

Switch on *Type-to-search*, hold any Alt or Shift+Alt dial and type a few letters: the dial becomes a search over every 3ds Max command. Enter runs the outlined best match.

Undo dial for big steps

Alt+7, aim NE or E: undo two or three steps in one move, by name.

Recent commands for repetition

Alt+8 repeats what you just did, newest first.

Selection sets under the hand

Shift+Alt+2 shows the scene's named selection sets as directions.

The viewport under one key

Alt+5 sets how the viewport is drawn; the mouse wheel turns to lighting, display and views - every switch on a direction or in the list under the dial.

Hotbox for the rare command

Alt+5 and one wheel notch away shows all of 3ds Max's menus - including other plugins' - without hunting through the menu bar. Ctrl+Shift+middle mouse can open it too, if you bind it.

Scripts on a dial

Custom script... accepts any MAXScript. Snapshot the selection first (`local picked = selection as array`) when the code changes objects - the built-in dials do.

Find the words for a condition

`MarkingForge.contextInfo()` tells you the exact class and modifier names with your object selected.

Print the cheat sheet

Cheat sheet... in the editor makes a PDF of your saved dials to keep beside the keyboard for the first week; it tells you the page count when it is done.

Share a setup safely

Save preset... leaves out paths and usage; Load preset... strips scripts unless you allow them.

One key, several tasks

Give a dial sets - *Modelling*, *UV* - and turn the mouse wheel while it is open to reach the others; a key on *Menu N - next set* changes which one the key opens. The same key and directions serve each task in turn.

A whole setup in one folder

Save every dial... before an experiment, **Load every dial...** to come back - every dial, its sets, the colours and every option, exactly as saved. Right-click a dial for the same.

Restore one dial, not all

Dial library... opens on the original of the dial you are editing - **Restore Menu N** is one click.

Groups in two strokes

A submenu holds a group the hand reaches by direction twice - see Create (Alt+4): Shapes S, More primitives SW, Helpers W, Cameras and lights NW.

Back up your dials

Copy `menus.json` - a plain text file - or use **Save every dial...**.

17. Problems and solutions

There is no MarkingForge menu after installing.

Restart 3ds Max - C++ plugins load only at start. Then type `MarkingForge.version()` in the Listener. If it says "Unknown property", check that `%APPDATA%\Autodesk\ApplicationPlugins\MarkingForge\PackageContents.xml` exists and reinstall the Microsoft Visual C++ 2015-2022 x64 runtime.

3ds Max reports a duplicate plugin or class at start.

Two copies of `MarkingForge.gup` are installed. Run the installer: it lists every copy it finds. Keep only the one in ApplicationPlugins.

Holding the key does nothing.

Open **Customize > Hotkey Editor** and search the key: another command may own it, or the dial has no key. Run **MarkingForge > Set up standard shortcuts...** or set it in the editor and press Apply shortcuts.

The key opens a dial, but it is the wrong one / wrong contents.

A variant matched. `MarkingForge.eventLog()` names the variant that opened. With *Menu under the cursor* on, the object under the cursor decides - move the cursor to empty space or switch the feature off.

A direction has a red caption.

The command does not exist in this 3ds Max - usually a plugin that is not installed, or a macro from another machine. Replace the item in the editor.

A dial opens empty.

It uses a live source with nothing to show: the modifier stack needs exactly one selected object, selection sets need named sets in the scene, the undo/recent dials need their experimental switch on. The cheat sheet lists empty dials.

Save says the configuration changed after it was opened.

menus.json was changed on disk while the editor was open (another editor window, a tick in MarkingForge > Experimental features, a page switched from a dial). Choose **Overwrite with my edits** to keep what the editor shows (1.5.55), or **Save a copy...**, press **Reload from file** and carry your edits over.

Chamfer / Inset opens its settings when I do not want them (or the other way round).

Move past the rim to reverse it once; use **Settings...** on the item or *showItemSettings* to change the default.

A flick at Extrude, Bevel or Inset always opens (or never opens) its settings.

A flick is measured against the same rim as the held dial: a flick longer than the way to the dotted circle counts as past it (since 1.5.2 - until 1.5.1 every flick counted as short). `MarkingForge.eventLog()` shows each flick's length as *[stroke N px]*; hold the key once to see where the rim lies on that dial.

Past the rim, Inset (Extrude, Bevel) does not change the mesh.

Without its settings window the command starts like its button in the Modify panel: drag in the viewport to set the amount. Release near the dial when you want the window instead.

An Edit Poly command does nothing.

Those commands act on the Edit Poly open in the Modify panel. Open the Modify panel with the modifier selected - the message in the Listener says so.

After work in panels and menus, a dial key does nothing until I click a viewport.

3ds Max's menu bar took the keyboard - a lone press of Alt does that. Since 1.0.9 MarkingForge disarms it while a dial is open and hands the keyboard back when the menu bar takes it right after a dial key. If it still happens: Esc or one click in the viewport, then **MarkingForge > Diagnostics > Event log** - the lines marked **FOCUS** say which window took the keyboard; send them with your report.

The set I switched to from the dial is gone after Save in the editor.

The editor was open while the switch wrote the file; its Save then refuses and offers a copy. Press **Reload from file** before editing on after a switch.

The dial stays on screen / a key feels stuck after switching programs.

Switching programs cancels the dial. If a key still seems held, press and release it once in 3ds Max.

The dial stayed open after I let go of the key, and the other dial keys did nothing.

3ds Max had switched its shortcuts off (after starting a Text object, for one) and did not report the release. Since 1.4.4 MarkingForge notices it within a quarter of a second and closes the dial as a release; `MarkingForge.eventLog()` shows **RELEASE_LOST**. If keys still do nothing afterwards, 3ds Max's own shortcuts are off too - click in a viewport or finish the command you started.

The mouse wheel does not turn the dial's pages - it zooms the viewport.

Only a dial with two sets or more has pages, and only at its first level - not inside a submenu, a hotbox or a search. A set added in the editor reaches the dial after Save. The footer says *page 1 of 2 - mouse wheel* when the wheel will work.

Typing letters while the dial is open does something unexpected.

That is *gestureSearch*. Esc closes the search; switch the feature off if you do not use it.

A mouse binding does not work or breaks navigation.

Bindings need a modifier and a dial with its own menu. `MarkingForge.mouseInfo()` shows what is bound. Pick another combination if it collides with a 3ds Max navigation shortcut.

I edited menus.json and it stopped working.

Diagnostics > Configuration status names the line with the error; only that item is disabled. **Reload from file** in the editor shows the state after your fix.

The editor says "Some of the file could not be read".

A dial in `menus.json` is not a JSON object (a hand edit gone wrong). The editor lists which dials it left out; saving from the editor removes them. Fix the file by hand first if you want to keep them - the plugin skips the same dials and reports them in Configuration status.

An update failed half-way.

The installer backs up the installed version first and puts it back when copying fails, so MarkingForge keeps working as before; its message says so. Free some disk space or pause the antivirus and run **Update / Repair** again.

The editor does not open.

The message shows the Python error. Check that the `tools` folder is inside the plugin folder; reinstall with **Update / Repair**.

I want my shortcuts back as they were.

Every change leaves `plugcfg\MarkingForge\hotkeys\backup_*.hsx`. Load one in the Hotkey Editor, or use **Undo** next to Apply shortcuts right after a change.

I want the original dials back.

Editor > **Load default dials...**, tick *Replace the slots that already have a menu as well* to reset all. For ONE dial: pick it, **Dial library...**, **Restore Menu N**. A setup you saved with **Save every dial...**: **Load every dial...** and the folder.

Over Track View (or Slate) the dial shows the page I turned to in Unwrap.

Fixed in 1.5.55: over a window with a variant of its own on another page, that page opens; the page you turned to stays remembered for the 3D view.

Load every dial... says it could not read the dials in the folder.

One file in it is damaged or is not a MarkingForge dial, and the whole load is refused rather than half done - nothing changed. The message says why (a file that is not a dial file is named). Put back a good copy of that file - the folder's list, `MarkingForge dials.json`, still names it - or choose another folder made by **Save every dial...**; one dial alone loads with **Load dial...**.

The test copy set with MARKINGFORGE_CONFIG_DIR is not used.

The variable must hold the FULL path of a folder that exists, and 3ds Max must be started after it was set. A relative path or a missing folder is ignored and the usual folder used - **Diagnostics > Configuration status** says which folder is in use and why.

Everything is broken and I want to start clean.

Close 3ds Max and rename `plugcfg\MarkingForge\menus.json`. The next start writes a fresh one with the built-in dials.

> **Still stuck?** Open an issue at github.com/Forge-Plugins/MarkingForge/issues or write to forgeplugins@gmail.com. **MarkingForge > Report a bug...** puts your 3ds Max and MarkingForge details on the clipboard, ready to paste.

18. Files, privacy and safety

What	Where
Plugin	`%APPDATA%\Autodesk\ApplicationPlugins\MarkingForge` (or `%ProgramData%\...` for all users). Each 3ds Max version has its own download and its own folder - MarkingForge (2027), MarkingForge_2026 (2026), MarkingForge_2025 (2025) - so they live side by side
Your dials	`%LOCALAPPDATA%\Autodesk\3dsMax\2027 - 64bit\ENU\en-US\plugcfg\MarkingForge\menus.json`
Hotkey set and backups	same folder, `hotkeys\MarkingForge_Hotkeys.hsx` and `hotkeys\backup_*.hsx`
Usage counters	same folder, `usage.json`
The last settings-dialog attempt	same folder, `caddy_diag.txt` - read by `MarkingForge.caddyDiag()`
First-start shortcuts	same folder, `setup_shortcuts.flag`, `skip_shortcuts.flag`, `shortcuts_first_start.done` (chapter 9)
The editor's own colours	same folder, `editor_colors.json` - only once you choose some
Your colour presets	same folder, `dial_color_presets.json` (dials) and `editor_color_presets.json` (the editor); one preset moves as a `.mfcolors` file
Another folder for the dials	set the environment variable `MARKINGFORGE_CONFIG_DIR` to the FULL path of a folder that exists (a test copy, a prepared studio set; 1.4.0): the plugin and the editor read and write menus.json, usage.json, the colour files and `caddy_diag.txt` there - NOT the `hotkeys\` folder, which stays in plugcfg\MarkingForge (1.4.7: 3ds Max keeps one active hotkey set per version, shared by all its sessions). Quotes round the path are taken off; a relative path or a folder that does not exist is ignored and the usual folder used (1.4.3). MarkingForge > Diagnostics > Configuration status says which folder is in use and why. The startup script's own files (the log and the first-start files) and the studio deployment script do not follow it - Deploy-MarkingForge uses -SettingsRoot.
Startup log	`plugcfg\MarkingForge\menu_log.txt`
The preset packs	`presets\MarkingForge_Pack_*.json` in the plugin folder (1.5.55) and `Presets\` in the download - Modelling, Animation, Look-dev, Archviz. **Load preset...** opens this folder
Your presets (Save preset...)	a `.json` file wherever you save it; the dialog offers `plugcfg\MarkingForge\MarkingForge_preset.json`, next to your dials
The dial library	`dials\` in the plugin folder (Built-in 24, Extra 12, Packs 15) and `Dials\` in the download - one `.mfdial` per dial
One dial (Save dial...)	a `.mfdial` file wherever you save it - plain JSON: `kind`, `version`, `name` and the `menu` (chapter 14)

Every dial (Save every dial...)	a folder *MarkingForge dials YYYY-MM-DD HH-MM* inside the one you choose: *NN name.mfdial* files and `MarkingForge dials.json`
A copy when Save meets a changed file	`menus.json.conflict.json` by default, wherever you choose

- MarkingForge sends nothing over the network.
- It changes shortcuts only when you ask, and backs them up first.
- It writes nothing into scenes unless you store a scene menu.
- It never rearranges your dials by itself.
- Its keyboard hook exists only while a dial is open and touches no key outside 3ds Max. The editor's **Set shortcut** window takes key presses only while it is in front, and only until it closes.
- Its mouse hook exists only while a dial is bound to a mouse button or has sets, and takes the wheel only when it turns a page.
- A **script** item runs with full rights, like any .ms or .mcr file - do not paste code you do not understand, and load presets from others with scripts removed (the default).

19. Support, bug reports and feature requests

ISSUES	github.com/Forge-Plugins/MarkingForge/issues
ROADMAP	trello.com/b/7gJVqG8Y/markingforge-features-roadmap
E-MAIL	forgeplugins@gmail.com
WEB	https://forgeplugins.tech (forgeplugins.tech - more plugins by the author)

Bug reports and feature requests are best opened as issues on GitHub - the forms there ask for exactly what is needed, and you can follow the fix. Questions, and anything you would rather not post publicly (licence, purchase), go to the e-mail address. Every message is read.

Reporting a bug

1. In 3ds Max choose **MarkingForge > Report a bug...** - it copies your 3ds Max version, MarkingForge version, configuration summary and start-up log to the clipboard. It works even when the plugin did not load.
2. Open github.com/Forge-Plugins/MarkingForge/issues and pick **Bug report** - or write an e-mail to forgeplugins@gmail.com instead. Both are links in the window that opens (since 1.5.20), one click each.
3. Say what you did, what you expected and what happened instead, then paste the details with **Ctrl+V**.
4. Add a screenshot if you can. When the problem is in one dial, attach a preset of it (editor > **Save preset...**).

Suggesting a feature

Open an issue with the **Feature request** form, choose **MarkingForge > Suggest a feature...**, or simply write. Describe what you would like to do and how you do it today - a concrete example from your work says more than a list of wishes. Ideas for new built-in dials, commands and options are all welcome. What is already planned and in progress is on the **Roadmap** above - **MarkingForge > Roadmap (Trello)** opens it too.

Nothing is sent by the plugin itself: the details go to the clipboard and your own e-mail program, so you see everything that leaves your computer. MarkingForge has no network access.